AN MD STUDIO GUIDE

Writing in Plain Text

A practical guide to writing in Markdown with MD Studio.

Dann Bleeker Pedersen

Generated 2026-10-26
md-studio.struktureretsundfornuft.dk

Contents

Foreword 3
Your First Document 5
Anatomy of the Editor 10
Structuring Documents 16
Formatting and Rich Content 22
Managing Files Locally 30
Exporting and Sharing 38
Writing as Practice 45
Appendix A: Keyboard Reference 52
Appendix B: Markdown Syntax Reference 58
Appendix C: Further Reading 65

Foreword

Markdown is plain text with a few conventions: a `#` in front of a heading, a `-` in front of a list item, two asterisks around a **bold** word. Any text editor on any computer made in the last forty years can open it, and there is no reason to think the next forty will be different.

So why a book, and why another editor?

Why another Markdown editor

Most Markdown editors make you choose. You can write the raw text and squint at the symbols, or you can write on a polished, word-processor surface that hides the symbols and now and then quietly rewrites them. The first is tiring to read. With the second, you are never quite sure what is in the file.

MD Studio shows **one document in two views**, side by side. On the left is the Markdown as written, character for character, the text that ends up in the file. On the right is the same document formatted, the way a reader will see it. You can type on either side, and the other side follows a moment later.

Day to day that is a small convenience, but it changes how you write. While drafting, you can stay in the formatted view and forget about syntax. When you want to check the structure, a glance at the raw text tells you which line is a heading and which is only bold, and if something looks wrong you can see why. When you edit in the formatted view, MD Studio keeps your original text for every block you did not touch, so a small change doesn't reshuffle the whole file.

The other reason is ownership. MD Studio runs in your browser, with **no server and no account**. The browser keeps your document while you work, and when you save, it becomes an ordinary `.md` file on your own disk. In Edge and Chrome, Save writes straight back to the file you opened, as a desktop app would. You can put that file in OneDrive, Dropbox or a git repository, open it in Notepad, or email it to someone who has never heard of MD Studio. The file doesn't depend on the app that wrote it.

That is what I want from plain text: files that stay yours, and that outlive whatever tool you happen to use this year, this one included.

Who this book is for

This book is for people who write things down and want them to last: meeting notes, project documentation, a README, the draft of a report, a reading log, the first chapters of something longer. If you have ever lost work to a proprietary format, an expired subscription or an app nobody maintains any more, you are the reader I have in mind.

You don't need to know Markdown already. Chapter 1 starts from a blank page, and the syntax arrives a little at a time, always next to what it looks like formatted. If you already write Markdown in another editor, skim the first two chapters for the places where MD Studio behaves differently and use the rest as reference.

You don't need to be technical either. MD Studio is a web page you can install as an app. There is nothing to configure before you start, and nothing in this book asks you to open a terminal. Git and sync folders come up as options; you can ignore them.

How the book is organized

Chapters 1 to 6 cover the tool.

- Chapter 1, Your First Document, gets a real file on your screen: the start screen, creating, opening and saving, and installing MD Studio as an app.
- Chapter 2, Anatomy of the Editor, explains the two panes and what it means that they are "synced". Read this one slowly, because the rest of the book builds on it.
- Chapter 3, Structuring Documents, is about headings, lists and sections, and keeping a document navigable as it grows.
- Chapter 4, Formatting and Rich Content, covers links, images, tables, code blocks and footnotes.
- Chapter 5, Managing Files Locally, goes further into where your document lives, what the browser may do with your files, and how to move work between devices.
- Chapter 6, Exporting and Sharing, is about getting a document out, either as the `.md` file itself or as a web page, PDF, Word document or plain text.

Chapter 7, Writing as Practice, covers the practice. It steps back from features to the habits plain text supports: when to reach for structure and when to just write, why Markdown and version control get along, and how switching between the two views can become a way of checking your own thinking.

The appendices are reference. Appendix A has every keyboard shortcut, Appendix B the Markdown syntax on one page, and Appendix C where to read more.

Read Chapters 1 and 2 in order. After that, jump to whatever you need. Every chapter quotes buttons and shortcuts exactly as the app shows them, so you can follow along with MD Studio open in another window. The book itself is written in Markdown.

Your document is kept as you type. Close the tab and it is still there tomorrow. Let's begin.

Your First Document

Opening, creating, saving, installing

Open MD Studio in your browser. You don't sign up for anything or download anything first. The first time you visit, you land on the **start screen**.

This chapter covers what you do around the writing: starting a document, giving it a name, opening files you already have, saving them back, and installing MD Studio so it behaves like the other apps on your computer. By the end you will have a `.md` file on your disk that you made yourself.

The start screen

At the top of the start screen is the name, **MD Studio**, and the tagline *Local-first Markdown, visual and raw side by side.* Below that is a row of large buttons:

- **Continue editing** takes you back to the document you were working on. It shows the file name and the document's first heading, so you can tell which one it is, and it only appears when there is something to continue.
- **New document** starts a blank page.
- **Open file…** lets you pick a Markdown file from your computer.
- **Try the sample document** opens a short tour of MD Studio and Markdown as a new, unsaved document, so you can experiment with it freely.

Under the buttons is the **Recent** list. The first time, it just says *Files you open or save appear here.* Once you have opened or saved a few files, it lists each one with its name, its first heading and when you last had it open. The end of this chapter comes back to it.

At the bottom is a hint, *Tip: drop a .md file anywhere to open it.*, and links to this book as a PDF and an EPUB.

You won't see the start screen every time. When you come back to MD Studio later, it opens straight into the documents you had open, where you left them. To get to the start screen from the editor, click **MD Studio** at the far left of the toolbar (on a narrow screen it just says **MD**).

Creating a document

Click **New document**. The editor opens with an empty page called `Untitled.md`, and that name appears at the top left of the toolbar.

On a wide screen the page is split in two. The left side is the **Markdown source**, the raw text. The right side is the **visual editor**, the same text formatted. Chapter 2 explains the two halves properly. For now, click in the left side and type:

```
# Shopping for the weekend

Things we need before **Saturday**:

- bread
- coffee
- something for the barbecue
```

A moment after you stop typing, the right side shows a large heading, a sentence with *Saturday* in bold, and a bulleted list. You typed plain text with a few symbols, and the right side shows what the symbols mean.

Look at the toolbar. A small dot has appeared next to `Untitled.md`. It means **unsaved changes**: the document has changes that are not yet in a file on your disk. Nothing asked you to save before you started, and you don't have to.

Your work is kept as you type

MD Studio keeps every open document in your browser, a fraction of a second after each change. Close the tab, close the browser or restart the computer, and when you open MD Studio again, `Untitled.md` is still there with the dot still showing.

Treat that as a safety net. A document that only lives in the browser is tied to that browser on that computer. If you want a file you can back up, share and open anywhere, save it.

Saving, and giving the document a name

Click **Save** in the toolbar, or press **Ctrl+S** (**Cmd+S** on a Mac). What happens next depends on your browser.

In Edge and Chrome on a computer, a normal save dialog opens with `Untitled.md` filled in as the name. Pick a folder, type a better name, say `weekend.md`, and click Save. A short message confirms *Saved weekend.md*, the toolbar shows the new name, and the dot is gone.

From then on, **Save writes straight back to that same file** without asking. Edit, press Ctrl+S, and `weekend.md` on your disk is updated in place, as it would be in a desktop app. This keeps working after you close and reopen MD Studio. The browser may ask you once to allow access to the file again; after that it saves silently.

Firefox, Safari and phone browsers don't let a web page write to your files. There, Save **downloads** the document under its current name, and the message says *Downloaded Untitled.md*. The file lands wherever your browser puts downloads, and each Save is a new download, so if you work this way, move the file to where it belongs and rename it there. Chapter 5 has more on working in these browsers.

Save as

Save as… (**Ctrl+Shift+S**) always asks where to save, even when the document already has a file. Use it to make a copy under a new name or to put your work in a different folder. After a Save as,

Save writes to the new file.

Save as is in the menu at the right of the toolbar, and in the command palette, which opens with **Ctrl+K** and holds every command. Chapter 2 introduces both.

Opening a file you already have

The usual way is the file picker. Click **Open file…** on the start screen or in the menu, or press **Ctrl+O**, and choose a file. It opens in Read view, as a page to read, with a short message, *Opened notes.md*. Press **Edit** to change it. The picker looks for Markdown files with the extensions `.md`, `.markdown`, `.mdown` and `.mkd`.

If the folder is already open on your screen, dragging is quicker. Drag a `.md` file (or a `.txt` file) from your file manager and drop it anywhere on the window. It opens in its own tab, wherever you let go. You can drop several files at once.

On Windows, once MD Studio is installed as an app (see below), you can open a file from Explorer: right-click it, choose *Open with*, and pick MD Studio. If Windows uses MD Studio for `.md` files, a double-click opens them there too.

In Edge and Chrome, a file you open with the picker or from Explorer stays connected to the file on disk, so Save writes back to it. A dropped file is a copy. It opens with its content, but the first Save asks where to put it, as it does for a new document.

Every file gets its own tab

Opening a file never replaces the document you are working on. Each new document or opened file gets its own tab, and as soon as two documents are open, a **tab bar** appears above the editor. Click a tab to switch to it. Each tab remembers its own cursor and scroll position.

These keys make tabs quicker:

Shortcut	What it does
Alt+PageDown	Next tab
Alt+PageUp	Previous tab
Alt+W	Close tab

They use Alt because the browser keeps Ctrl+W and Ctrl+Tab for its own tabs. You can also click the **×** on a tab. If the tab has unsaved changes, MD Studio asks *Discard unsaved changes?* first. Choose **Discard** to close it anyway, or **Cancel** to go back and save. Closing the last tab takes you back to the start screen.

If you open a file that is already open, MD Studio switches to its tab instead of opening a second copy.

The Recent list

The **Recent** list on the start screen grows as you open and save files. Click an entry to reopen it. Each entry is marked in one of two ways:

- **File on disk** means MD Studio still has a connection to the real file (Edge and Chrome only). Clicking it opens the file's current content, even if you changed it in another program in the meantime, and Save writes back to it. The browser may ask you to allow access first.
- **Saved copy** means MD Studio opens the copy it kept of the document. You see this in other browsers and for dropped files. It also happens when the file has moved or been deleted, or when you decline access, and MD Studio then tells you it opened the saved copy.

The **×** at the end of a row removes that entry, and **Clear list** removes them all after asking *Clear the recent list?* Neither touches the files themselves.

Installing MD Studio as an app

You can install MD Studio like an app, with its own window and icon and no address bar. An installed MD Studio opens like your other programs and works offline, and shortcuts the browser would otherwise take, such as **Ctrl+1** to **Ctrl+3**, go to MD Studio instead.

Edge and Chrome offer **Install app** in the address bar while MD Studio is open. Confirm it, and MD Studio opens in its own window from then on. On Windows, the installed app is also what makes *Open with* in Explorer possible.

On a phone or tablet, whether and how you can add MD Studio to the home screen depends on the browser; look in its menu for *Add to Home Screen*. On a phone, open documents from inside the app with **Open file…**. Split view stacks the panes one above the other on a narrow screen.

Once MD Studio has everything it needs, it shows the message *MD Studio is ready to work offline.* After that it starts without a connection.

How updates arrive

You never download an update by hand. When a new version is published, MD Studio fetches it in the background the next time it runs with a connection. When the update is ready, the message *A new version of MD Studio is ready.* appears with a **Reload** button. Click it when it suits you, and the app restarts on the new version with your documents as you left them. If you ignore the message, nothing changes under your cursor; the new version takes over once you have closed MD Studio and opened it again.

Now you try

Make a real file, start to finish, in five minutes.

1. From the start screen, click **New document**. Type a heading line starting with `#` , a sentence, and a short list where each line starts with `-` . Watch the right side format it, and watch the dot appear next to `Untitled.md`.
2. Close the browser tab and open MD Studio again. The document is still there, dot and all.
3. Press **Ctrl+S** and save it under a proper name in a folder you will find again. Check that the dot

is gone and the toolbar shows the new name.

4. Change a word and press **Ctrl+S** again. In Edge or Chrome, open the file in Notepad or TextEdit: your change is there, and the file is just text.
5. Click **MD Studio** in the toolbar to go to the start screen. Your file is in the **Recent** list. Drop another `.md` file on the start screen and watch the tab bar appear.
6. Click **Try the sample document**. It is a short tour of the editor and of Markdown, and a good scratch pad for the next chapter.

The next chapter looks at what happens between opening and saving: the two panes, and what it means that they are synced.

Anatomy of the Editor

The split view, the two panes, and what "synced" means

You can write in MD Studio without reading this chapter. Type on either side, press Save, and it works. But the editor is built around one idea, two views of a single document, and once that is clear the rest makes sense: why the right side updates a moment after the left, why your formatting survives a visual edit, and why the panes scroll together the way they do.

Open a document to follow along. The sample document from the start screen (**Try the sample document**) works well, because it has headings, lists, a table and a code block.

The toolbar

The toolbar runs across the top of the window. At the far left, **MD Studio** takes you back to the start screen (on a narrow screen the button just says **MD**). Next to it is the file name, for example `weekend.md`. A dot after the name means **unsaved changes**: what you see differs from what was last saved to disk. The browser keeps the document either way (see Chapter 1), so the dot only tells you about the file.

The right-hand side depends on whether you are reading or editing. A file you open starts in Read view, and there the main button is **Edit**, next to **Aa** (reading controls) and **Focus**. While you edit, the main button is **Save**, with **Read** beside it to go back to reading, and a switch with three layouts, **Split**, **Markdown** and **Visual**, with the current one pressed. Two small icons show the folder panel and the **Outline**, a panel listing the document's headings.

Everything else is in the menu at the far right: **New document**, **Open file…**, **Open folder…**, **Save as…**, **Export…**, **Commands** (the command palette), **Keyboard shortcuts**, **Settings** and the user guide. Chapter 1 covers New, Open and Save. Export saves a copy in another format and has Chapter 6 to itself. On a phone the menu also holds the layouts and the panels, so the toolbar keeps only the file name, Edit or Read, Save and the menu. Press **?** whenever no text is being typed to see every keyboard shortcut.

When two or more documents are open, a tab bar sits between the toolbar and the editor. Each tab shows the file name and, if the document has unsaved changes, the same dot.

The text pane

In split view, the left side is the **Markdown source**: the document as it is stored in the file, every `#`, `*` and `-` included. Nothing here is hidden or converted. Open the file in Notepad and you see the same characters in the same places.

To make it easier to read, the text pane colours the Markdown as you type. Heading markers, emphasis, links and code stand out from ordinary text, and the colours follow the light or dark theme. Inside a fenced code block, the code is coloured by language when you name the language after the opening fence:

````
```js
const greeting = 'Hello, Markdown';
```
````

By default the text pane shows **line numbers** in the margin and **wraps long lines** to fit the pane, so a long paragraph doesn't run off to the right. You can turn either off in Settings.

Use the text pane when you need precision: fixing a stray character, lining up a table, choosing how a list is marked, or pasting in Markdown from somewhere else.

The visual pane

The right side is the **visual editor**. It shows the same document formatted, with headings as headings, tables as grids and links as links. It looks like a finished page, but you can edit it directly, as in a word processor. Click into a paragraph and type, press Enter at the end of a list item to start the next one, or press **Ctrl+B** for bold and **Ctrl+I** for italic.

Above the visual pane is a small **Formatting** toolbar. It has a **Text style** menu (paragraph, or heading 1 to 4), then buttons for bold, italic, strikethrough, inline code, link, bulleted and numbered lists, quote, code block, table and horizontal rule. A button shows as pressed when the selection already has that formatting. Chapter 4 goes through them.

The visual pane is good for drafting. When you are trying to get a thought down, the symbols get in the way, and here you don't see them.

Undo where you made the change

Ctrl+Z undoes and **Ctrl+Y** redoes, in whichever pane has the cursor. Undo works most predictably in the pane where you made the change. If you make a word bold on the right, Ctrl+Z on the right takes the bold away again, and the left follows.

Reading and three ways to edit

Read view and the three editing layouts decide which panes you see.

View	Shortcut	What you see
Split	Ctrl+1	Markdown source on the left, the formatted document on the right
Markdown	Ctrl+2	Markdown source only
Visual	Ctrl+3	Formatted document only
Read	Ctrl+4	Formatted document only, not editable, laid out for reading

The command palette has the same four as *View: split*, *View: Markdown only*, *View: visual only* and *View: read*. In Visual view, Ctrl+click follows a link. In an ordinary browser tab, Ctrl+1 to Ctrl+4 switch the browser's own tabs, so use the toolbar or the palette there. In the installed app (Chapter 1) the shortcuts work.

Read view is for reading rather than writing. The document cannot be changed there: typing, paste, undo and the replace half of Find are all switched off, and the page is set in a narrower, more open column. Links work with a plain click. A web link opens in a new tab, a `#heading` link scrolls to its heading, and a link to another note, `[text](other.md)` or `[[Other note]]`, opens that note in a tab. Notes are found through the attachments folder, so choose it first (the notice above the editor offers to). In Visual view the same links open with Ctrl+click.

Every heading in Read view has a small arrow that folds its section away, every code block has a Copy button, and a click on an image shows it full size until you press Escape or click again. A file that changes on disk, say because another program saved it, reloads by itself as long as you have no unsaved changes. Save is left out of the toolbar and the palette in Read view, and Save as stays. In its place is **Edit**, which switches to the editing view you used last.

The **Aa** button in Read view opens the reading controls: the width of the column, serif or sans, line spacing, and a sepia page. **Focus** hides the toolbar and tabs so only the page remains. Every file you open starts in Read view, and a new document starts in the editing view you used last; **Open files in Read view** in the Settings dialog turns that off. MD Studio remembers where you stopped in each file. Rest the pointer on a link to another note to see the start of that note, and on a footnote reference to see the footnote. A heading shows a small # that copies a Markdown link to its section, and a front matter block shows as a card with the title, tags and fields. The keys for reading are in Appendix A.

Switching views only hides a pane. Both editors keep running, so switching is instant, and a pane you bring back is already up to date with what you typed in the other.

On a narrow screen such as a phone, split view stacks the two panes, source above and formatted below.

The view MD Studio starts in is a setting, **Default view mode**, and it is Read view unless you change it. If you mostly write in one pane, set it there and you will rarely touch the switch.

What "synced" means

There is one document, and the two panes are two views of it. MD Studio doesn't copy two documents back and forth. It holds one Markdown text, the same text that is saved to your file. The left pane shows that text as it is, and the right pane shows what it means. An edit in either pane changes that one text, and the other pane catches up.

When you type in the text pane, the visual pane updates shortly after you pause, in roughly a sixth of a second. It redraws only the part around your change, which keeps it quick on long files. A text edit never comes back to you "tidied up" by the visual pane: what you type on the left is what ends up in the file.

When you edit in the visual pane, the change is written back into the Markdown a fraction of a second later. The text pane changes only the characters that differ, so your cursor and scroll position on the left stay where they were.

Try it with the sample document in split view. Put the cursor on a word on the right and press **Ctrl+B**

, and `**` appears on both sides of the word on the left. Then type `## A new section` on an empty line on the left, and a heading appears on the right.

Why a visual edit can tidy formatting

Markdown often has more than one way to write the same thing. A bullet can start with `-`, `*` or `+`. A top-level heading can be a line starting with `#`, or a line underlined with `===`. These look identical in the visual pane, because they mean the same thing.

The visual pane works with meaning, not characters. After an edit there, it writes the document back out as Markdown in its own style. Left alone, that would rewrite every block in the file the first time you touched the visual pane, which hurts if you have a house style, or if you keep your notes in git and expect a one-word change to show up as a one-line difference.

So after each visual edit, MD Studio compares the new Markdown with what was there before and **keeps your original text for every block you didn't change**. Only the block you edited, usually one paragraph, list or heading, is written in the editor's style, and that may change its markers or spacing from how you wrote it.

For example, suppose your file contains:

```
Title
=====

* star bullet
* another

__strong__ and *em*

Last paragraph here.
```

In the visual pane, you click at the end of the last paragraph and type `Edited`. The heading with its `=====` underline, the `*` bullets and the `__strong__` paragraph come back exactly as you wrote them. Only the last paragraph has changed.

MD Studio also checks each merge. If it can't be sure that keeping your original text gives the same document, for instance when links are defined elsewhere in the file and an edit could change what they point to, it uses the visual pane's version throughout.

This matters when you care about the characters as well as the result: a file under version control, a document with a house style for lists or emphasis, or Markdown that another tool reads. For those, make careful edits in the text pane, where nothing is rewritten for you. When you are just writing, use whichever pane suits you.

Linked scroll

In split view the two panes scroll together, so the paragraph you are reading on one side is also in view on the other. This is **linked scroll**, and it is on by default.

The two panes are very different heights. A table is a handful of short lines in the text pane and a

tall grid on the right; an image is one line of Markdown and hundreds of pixels of picture. If the panes scrolled the same distance, or the same percentage, they would drift apart within a page or two.

So linked scroll **follows headings, not pixels**. MD Studio knows where every heading is in each pane. When you scroll, it works out which section you are in and how far through it, say 40 percent of the way through *Budget*, and scrolls the other pane to the same point in its own *Budget* section. At each heading the panes line up exactly, and between headings they move proportionally. Scroll to the bottom of one pane and the other goes to its bottom too.

The pane you are in leads. Whichever pane has the mouse pointer over it, or the cursor in it, drives the other. Move the pointer to the other side and that side takes over, so the two never fight over the scrolling.

Linked scroll therefore works best in a document with headings, and the more evenly they are spread, the closer the panes stay. A long document with no headings has only its top and bottom to line up. Chapter 3 has more on headings, which are useful for much more than scrolling.

To turn linked scroll off, choose *Toggle linked scroll* in the command palette, or clear **Scroll both panes together in split view** in Settings.

The outline

Outline (the list icon in the toolbar), or **Ctrl+Shift+O**, opens a panel listing the document's headings. Click one to bring that section to the top of both panes. The section you are reading is highlighted as you scroll. On a narrow screen the outline slides over the editor instead of sitting beside it.

The command palette

Most of what MD Studio can do is also a command, and the **command palette** lists them. Open it with **Ctrl+K**, **Ctrl+Shift+P** or **Commands** in the menu. The box says *Type a command, or # to jump to a heading*.

Start typing and the list narrows to matching commands, then press Enter to run the highlighted one. A few letters is usually enough: `save a` finds *Save as…*, `split` finds *View: split*, and `pdf` finds *Export as PDF*. Commands that have a shortcut show it beside the name, which is an easy way to pick up the keys.

Type `#` instead, and the palette lists the document's headings. Add a few letters to narrow them down, and Enter jumps to that section. *Go to heading…* in the palette does the same. In a long document this is often the fastest way to move around.

Some commands are only in the palette, such as *Toggle linked scroll*, *Open the sample document*, *Go to start screen* and *Print*. Appendix A lists every shortcut, and **?** shows them in the app.

Settings

Settings in the menu, or **Ctrl+,**, opens a short dialog in sections. **Appearance** has the **Theme** (**Match system**, **Light** or **Dark**) and the **Text size** (**Small**, **Medium** or **Large**, for both panes). **Reading** has **Open files in Read view** (on by default) and **When the app starts, show**: **Read** (the default), **Split**, **Markdown** or **Visual**. **Editing** has **Wrap long lines in the Markdown pane** and **Show line numbers in the Markdown pane**, both on by default, and **Scroll both panes together in split view**.

More settings folds away the rest: how links to attachments are written and where pasted files go (see Chapter 7), whether web images load, and whether front matter goes into exports.

Changes apply immediately. Click **Done** to close the dialog. The browser keeps your settings, so they are still there next time.

Now you try

Open the sample document in split view and see the one document behind the two views for yourself.

1. Type a new paragraph in the text pane and stop. Count to one, and the visual pane has it.
2. Click into that paragraph on the right, select a word, and press **Ctrl+B**. On the left the word now has `**` around it, and your cursor in the text pane hasn't moved.
3. Watch the style rule at work. On the left, add a heading written with an underline: a line `Notes`, and under it a line `=====`. The right side shows an ordinary heading. Now type a word into a different paragraph on the right, and look left: your `=====` underline is still there, because you didn't edit that block in the visual pane.
4. Scroll slowly through the visual pane and watch the text pane keep pace, lining up at each heading. Move the pointer to the left and scroll there, and the left leads.
5. Press **Ctrl+K**, type `#`, and jump to the last heading. Then press **Ctrl+2**, **Ctrl+3** and **Ctrl+1** (in the installed app, or use the toolbar in a browser tab), and notice that nothing reloads.
6. Open **Settings** and try a different text size and theme. Turn line numbers off, and decide whether you miss them.

From here on the book is about what you put in the editor, starting with how to give a document a structure that holds up as it grows.

Structuring Documents

Headings, lists and sections, and keeping a document navigable as it grows

A short note needs no structure. You write it, read it, and move on. Documents grow, though. The meeting note becomes a project brief, the brief becomes a plan, and one day you spend half a minute scrolling for the paragraph about the budget.

Markdown gives you headings, lists and blank lines to deal with this. MD Studio reads that structure back to you through the outline, the heading jump in the command palette, and linked scroll, and all three work only as well as your headings do.

Headings are the outline

A heading is a line that starts with one to six # characters and a space. The number of # signs is the level.

```
# Office move 2027

## Why we are moving

## Options

### Stay and refit

### Move to the harbour site

## Timeline
```

The visual pane renders this as a large title, three section headings, and two smaller sub-headings under *Options*. The Markdown pane reads almost the same way, because the hash marks are the structure and you can see them in both views.

Most documents do well with one H1 at the top for the title (one in total, not one per section), H2 for the sections you would list in a table of contents, and H3 now and then when a section has parts of its own. If you find yourself reaching for H4, ask whether that section wants to be its own document. The end of this chapter comes back to that.

Do not skip levels. An H4 directly under an H2 renders fine, but it tells the reader a layer is missing, and the outline shows the gap as an odd jump in indentation.

Keep headings short and write them as labels. *Timeline* is a good heading. *Here is when we think everything will happen* is a paragraph in a heading's clothes. Someone skimming the outline should be able to guess what is under each label.

Making headings in either pane

In the Markdown pane you type the hash marks. In the visual pane, put the cursor in a paragraph and pick a level from the **Text style** menu at the left of the format toolbar: **Paragraph**, **Heading 1**,

Heading 2, **Heading 3** or **Heading 4**. Choosing **Paragraph** turns a heading back into ordinary text. The editor library behind the visual pane is also set up to convert `##` typed at the start of an empty line into a heading.

Levels 5 and 6 exist in Markdown but are not in the menu. If you need them, type them in the Markdown pane, and then ask yourself again whether you need them.

The other kind of heading

Markdown has a second, older way to write the first two levels: underline the text with `=` for level 1 or `-` for level 2.

```
Office move 2027
================

Why we are moving
-----------------
```

These are *setext* headings, and the `#` style is called *ATX*. Both are standard, both render the same, and MD Studio recognises both, so a file written with setext headings needs no converting. For your own writing, ATX is the better habit. It covers all six levels, and a line of dashes has another meaning (see *Horizontal rules* in Chapter 4).

What MD Studio counts as a heading

The outline, the heading jump and linked scroll all read the same list of headings. MD Studio finds **top-level ATX and setext headings**, the ones that sit directly in the document and that a renderer turns into section titles. It skips a `#` line inside a fenced or indented code block (a shell comment, say), a heading written inside a block quote or a list item, and `#hashtag` with no space after the hash, which is just a paragraph.

So if a heading you expected is missing from the outline, check whether it has ended up inside a quote or a list, or whether the space after the `#` is missing.

Seeing the shape: the outline

Press **Ctrl+Shift+O**, or click **Outline** in the toolbar, and a panel opens beside the editors with every heading in the document, indented by level. Click one and that section comes to the top of both panes. As you scroll, the heading of the section you are reading is highlighted, so the panel also tells you where you are. On a narrow screen the outline covers the editors instead of sitting beside them. In a document with no headings yet, the panel says *Headings appear here.*

Reading the outline top to bottom, without the body text, is the quickest structural review there is. Does it tell the story of the document, in the order a reader needs it? You will see problems there that you cannot see from inside the prose.

When you know where you want to go, the command palette is faster. Press **Ctrl+K** and type `#`. The palette, whose placeholder reads *Type a command, or # to jump to a heading*, switches from

commands to headings. Type a few letters of the heading and press **Enter**. The matching is forgiving, so `#harb` finds *Move to the harbour site*. If you would rather pick a command, **Go to heading…** in the palette does the same thing.

Lists: sets, sequences and to-dos

Markdown has three kinds of list, and each says something different about its items.

A bulleted list is a set: the items belong together and their order is not the point. Start each line with `-`, `*` or `+` and a space.

```
- Meeting rooms
- Bike storage
- Parking
```

A numbered list is a sequence. Use one when the order matters, as in steps, rankings, or a procedure someone will follow.

```
1. Give notice on the current lease.
2. Sign the new lease.
3. Book the movers.
```

Markdown takes the number of the first item and counts on from there. You can write every item as `1.` and let the renderer number them, which saves renumbering when you insert a step in the middle. A list that starts at `4.` renders as 4, 5, 6, handy when a list picks up again after a paragraph of explanation.

A task list is a list of to-dos. It is a GitHub Flavored Markdown extension: a bulleted item that starts with `[ ]` or `[x]`.

```
- [x] Measure the new floor plan
- [ ] Get quotes from three movers
- [ ] Tell the post office
```

The visual pane shows each item with an empty or ticked box. To tick an item off, change `[ ]` to `[x]` in the Markdown pane. That one character holds the whole state, which is why task lists survive any tool that reads text.

Nesting

Indent a list item under another to nest it. Line the child up with the text of its parent: two spaces under a `-` bullet, three under `1.`.

```
1. Pack
   - Label every box with a room
   - Keep one box of essentials
2. Move
```

Nesting is structure in the same way headings are. Two levels are common and three are sometimes right. At four, the list usually wants to be a section with a heading.

Tight and loose lists

A list with no blank lines between its items is *tight*, with each item on its own compact line. Put a blank line between items and the list becomes *loose*: each item is rendered as a paragraph, with more space around it. Tight lists suit short items, and loose ones suit items that run to several sentences. You can mix the two in one list, but you rarely mean to.

Lists from the toolbar

In the visual pane, **Bulleted list** and **Numbered list** in the format toolbar turn the current paragraph into a list. The visual editor is also set up to start a list when you type `-` or `1.` at the start of a line.

Blank lines are structure too

A blank line ends a paragraph. A single line break inside a paragraph does not:

```
The lease ends in March.
We have six months to decide.

The new site is available from January.
```

This renders as two paragraphs, and the first one reads *The lease ends in March. We have six months to decide.* on a single line. That means you can break lines in the Markdown pane wherever you like without changing the result. One sentence per line is a popular habit, because it makes changes easy to compare. When you do want a line break inside a paragraph, end the line with a backslash.

Blank lines also keep blocks apart. Put one before and after every heading, list, quote, table and code block, and two blocks will never merge into one by surprise.

Sections as movable units

In Markdown, a section is a heading and everything after it up to the next heading at the same or a higher level. In the example at the top of this chapter, the *Options* section includes both H3 subsections and stops at *Timeline*. Nothing opens or closes a section. The heading marks the boundary.

That makes sections easy to move. Work in the Markdown pane: select from a heading down to the line before the next heading at its level, cut, put the cursor at the start of the heading it should go before, and paste. The outline gets you to both places quickly. The section is plain text, so its subsections go with it.

This works best when each section stands on its own. A section that says *as described above* breaks when it moves, so name the thing instead: *as described under Options*. Heading levels matter here too. A sub-point written as an H2 when it belongs under another H2 will not move with its parent.

Find and replace helps with structure as well. Press **Ctrl+F** in the Markdown pane, switch on

Regular expression (the `.*` button) and search for `^##` to step through every H2, with a count of how many there are. Searching for `^#{4,}` finds every heading at level 4 or deeper, a quick check on whether the document has grown too deep.

Headings double as scroll anchors

In split view, linked scroll keeps both panes on the same part of the document. It goes by headings rather than pixels, because the two panes render the same text at very different heights. A table is a few short lines in the Markdown pane and a tall grid in the visual one, so matching pixel positions would drift apart within a page. MD Studio works out which section you are in and how far through it, and puts the other pane at the same point. Chapter 2 covers how to switch it on and off.

In practice, your headings are the anchors. Between two headings the panes follow each other proportionally, which is close enough over a few screens of text. A long stretch with no headings gives linked scroll nothing to hold on to, and the panes can drift noticeably out of step in the middle of it. When that happens, the stretch usually needs a heading anyway.

Visual edits and your source style

Markdown often has several spellings for the same thing: `-` or `*` for bullets, `**` or `__` for bold, ATX or setext for headings. When you edit in the Markdown pane, what you type is what is saved.

When you edit in the visual pane, MD Studio rewrites **only the blocks you changed**, and those blocks come out in the visual editor's own style: `-` bullets, `**` for bold, ATX headings. Every block you did not touch keeps its original text exactly, spacing included. If your file uses `*` bullets and you add an item to one list in the visual pane, that list switches to `-` and the rest of the document stays as it was.

Usually this does not matter. If you want a consistent source style, for small git diffs or a colleague's house style, make those edits in the Markdown pane. Chapter 2 explains why the trade-off exists.

When a document has outgrown its structure

A structure that worked at two pages can be a maze at twenty. Watch for an outline that no longer fits on the screen, so that you scroll the table of contents to find the table of contents. H4s are another sign. They are not always wrong, but they often mean one section has become a document of its own. So is a single section that holds most of the text, with the other headings pointing into one sprawling room. A section called *Misc*, *Other* or *Notes* is a drawer things fall into, and it only grows. And if you have started using Find to get around, searching for a word you know is in there, the headings have stopped telling you where things are.

The first fix is to restructure in place. Read the outline and rename headings until they say what is under them. Promote a sprawling H3 to an H2, merge two thin sections, and empty the *Misc* drawer into the sections where its contents belong.

The second fix is to split. Pick a section that stands on its own, cut it, and start a **New** document (**Alt+N**). Paste it in, promote its heading to the H1, and save it next to the original under a clear

name. Both documents stay open as tabs, so you can go back and forth while you tidy the seams. Then leave a pointer where the section used to be:

```
## Budget

The full budget is in [budget.md](budget.md).
```

This is a relative link. It names a file in the same folder, so it keeps working wherever the folder goes, and GitHub and most Markdown tools follow it. Chapter 5 has more on keeping a folder of related documents together.

Once you have several files, a short index document that links each of them, with a line of explanation apiece, gives the set the outline that a single document gets from its headings.

Now you try

Open a document of your own, or start one with three or four sections you know well, such as a project, a trip or a recipe collection.

1. Give it one H1 and make every section an H2. Open the outline with **Ctrl+Shift+O** and read only the headings. Rename any that do not say what is under them.
2. Add a bulleted list, a numbered list and a task list where each fits the content. Tick one task off by changing `[ ]` to `[x]`, and look at its box in the visual pane.
3. Press **Ctrl+K**, type `#` and a few letters, and jump to your last section.
4. Move one section above another by cutting and pasting it in the Markdown pane. Check the outline to see that it moved whole.
5. Search for `^##` with **Regular expression** on, and count your sections.
6. If one section is pulling away from the rest, split it into its own file and link to it.

The next chapter covers what goes inside the sections: emphasis, links, images, tables and code.

Formatting and Rich Content

Links, images, tables, code blocks, footnotes

Structure gives a document its shape. This chapter covers what goes inside it: a word that needs weight, a link to a source, a screenshot, a table comparing a few options, a snippet of code that has to be copied exactly.

Each of these is a small piece of punctuation that you can read in the Markdown pane and that the visual pane renders as something richer. If the source stops being readable as text, you are probably using more formatting than the writing needs.

Emphasis

Wrap words in one asterisk for italic and two for bold:

```
The lease ends in *March*, not April. Deposits are **not refundable**.
```

The visual pane shows *March* in italic and **not refundable** in bold. Underscores work too (`_March_`, `__not refundable__`). Asterisks are the safer habit, because Markdown ignores underscores inside words on purpose so that `snake_case_names` come through as written.

Strikethrough is a GitHub Flavored Markdown extension. Wrap the text in double tildes, `~~like this~~`, and it renders crossed out. In a running document it shows that a decision changed: ~~Friday~~ Thursday.

In the visual pane, **Ctrl+B** and **Ctrl+I** toggle bold and italic on the selection, and the format toolbar has **Bold**, **Italic** and **Strikethrough** buttons. A button shows as pressed when the selection already has that formatting, and clicking it again removes it. The visual editor is also set up to turn typed asterisks into formatting, so `**urgent**` should turn bold as you type the closing pair.

Use emphasis the way you would raise your voice, which is to say rarely.

Inline code

Backticks mark a fragment as code, or as anything that has to be read literally, such as a file name, a command or a setting.

```
Save the file as `move-plan.md` and run `git add move-plan.md`.
```

The fragment renders in a monospaced font, and nothing inside it counts as formatting, so `*not italic*` keeps its asterisks. To show a backtick itself, wrap the fragment in two backticks with a space inside: `` `code` `` shows `` `code` ``. In the visual pane, **Inline code** in the toolbar marks the selection.

Links

A link is the visible text in square brackets followed by the address in parentheses:

```
See the [floor plan](https://example.com/floorplan) before Friday.
```

The visual pane shows *floor plan* as a link. You can add a title in quotes after the address, and most renderers show it as a tooltip: `[floor plan](https://example.com/floorplan "Ground floor, rev. 3")`.

To make a link in the visual pane, select the text and click **Link** in the format toolbar. A small dialog asks for the address, starting from `https://`. Type or paste it and choose **Apply**. With the cursor in an existing link, the same button shows the current address so you can change it, and the field's label tells you how to remove the link: *Link address (leave empty to remove the link)*. Clear the field and apply, and the text stays while the link goes.

Links to other files

An address does not have to start with `https://`. A **relative** link names a file next to the document:

```
The numbers are in [budget.md](budget.md), the photos in [images](images/).
```

Relative links keep working when the whole folder is moved, zipped, emailed or pushed to GitHub, so they are the way to connect a set of documents (see Chapter 3 on splitting documents).

Autolinks

When the address is the text you want to show, put it in angle brackets: `<https://example.com>`. GitHub Flavored Markdown also turns bare addresses into links by itself, so `www.example.com` and `https://example.com` in running text are both clickable once rendered. The angle bracket form works for email addresses too: `<someone@example.com>`.

Reference links

In a paragraph with many links, you can move the addresses out of the way:

```
The [lease][1] and the [insurance terms][2] both changed.

[1]: https://example.com/lease
[2]: https://example.com/insurance
```

The definitions do not render. They only supply the addresses. Inline links are easier to move around, so use references only where they make the paragraph easier to read.

Images

An image is a link with an exclamation mark in front. The text in the brackets is the *alt text*, which a screen reader reads out and which shows if the image cannot be loaded.

```
![Ground floor plan with the new meeting rooms](images/floorplan.png)
```

Write alt text that says what the picture shows. *Image* or *screenshot* tells a reader who cannot see it nothing.

Pasting and dropping files

You rarely need to type image links by hand. Paste an image with **Ctrl+V** into either pane, or drag a file from your desktop onto one. What happens next depends on where the document lives.

In Edge or Chrome, with the document saved on disk, the first image you add brings up a question: *Save images next to the document?* (Any other kind of file, such as a PDF, does not ask. Until a folder is chosen it shows a notice and inserts nothing, so choose one first with **Choose attachment folder…** in the command palette, or paste an image to be asked.) Choose **Choose folder…** and pick the folder the document is in (the folder picker opens there). If the document belongs to an Obsidian vault, pick the vault's top folder instead; it contains the note, and MD Studio then follows the vault's own settings. Without a `.obsidian` folder, every image you paste or drop is saved as its own file in an `images` folder beside the document, named by date and time such as `image-20261002-171530.png`, and linked with a relative path:

```
![floorplan](images/image-20261002-171530.png)
```

The alt text starts out as the original file name, so replace it with a real description. If you pick a folder that does not contain the document, MD Studio tells you so and embeds that image, and the next image asks again. If you choose **Cancel**, the image is embedded and MD Studio stops asking for that document for the rest of the session.

In a vault the file goes where the vault's attachment setting says, which may be the vault's top folder, a folder next to the note or a fixed folder such as `Attachments`. A pasted image is named the way Obsidian names it, as in `Pasted image 20261012093000.png`, and a dropped file keeps its own name with anything unsafe replaced. If the name is taken, MD Studio adds a number rather than overwriting. Any kind of file can be dropped this way, not only images: a PDF, a recording or a spreadsheet is saved and linked.

How the link is written follows **Link style** in Settings. *Wikilinks* writes `![[Pasted image 20261012093000.png]]`. *Markdown links* writes `![](Pasted%20image%2020261012093000.png)` for images, PDFs, audio and video, and `[name](path)` for other files. *Auto (follow the vault)* does what the vault's own settings do, and uses Markdown links when there is no `.obsidian` folder, which is why documents that were fine before still work the same way. The vault's choice of shortest, relative or absolute paths is followed too. **Attachment location** in Settings can force the old behaviour, an `images` folder next to the note, even inside a vault.

When a file cannot be saved, MD Studio says why. After a reload the browser may want permission for the folder again, and the bar above the editor has an **Allow access** button; a file dropped from outside the browser can fail for the same reason, and the toast then points to that bar. With no folder chosen, or in a browser that cannot save to folders, an image is embedded in the document

instead and a file of another kind is not added.

In Firefox, Safari, on phones, or before you have saved the document, the image is embedded in the document itself. MD Studio first shrinks it so the longest side is at most 1600 pixels, then writes it into the Markdown as a long encoded address beginning `data:image/`. The document then carries its pictures inside it. That is convenient for a single file, but it makes the source long and hard to read. A large image brings up a notice that the document is getting big, with the reminder that saving it to disk lets images go in a folder.

Obsidian embeds

Notes from Obsidian use `![[plan.png]]` to show a file and `![[Other note]]` to show another note inside this one. MD Studio shows these in the visual pane and leaves the text exactly as written, also when you edit the paragraph around it. Pictures take a size (`![[plan.png|300]]`), PDFs a page (`![[report.pdf#page=3]]`), and audio, video and other files appear as a player or a labelled chip with an **Open** link. An embedded note appears as a quoted block that you read but cannot edit here; change it in its own note. Plain links such as `[[Other note]]` show as a chip, and clicking one does nothing yet. Appendix B lists every form, and Chapter 5 explains how MD Studio finds the files.

Until a folder is chosen, or where the browser cannot read folders, each embed is a labelled placeholder with the original text beside it. Nothing is lost, and a bar above the editor says what to do. Embeds written inside code are shown as code and never counted.

Files next to the document, or web addresses

Images can also come from the web: `![Logo](https://example.com/logo.png)`. That keeps the document small, but the picture only shows while you are online, and only as long as the other site keeps it at that address.

Where the image lives	Good for	Watch out for
images folder next to the document, or the vault's attachment folder	Documents on disk, folders in git, Obsidian vaults, anything you share as a folder	Send the folder, not just the .md file
Embedded in the document	A single self-contained file; browsers without folder access	Long, unreadable source; larger files
A web address	Images that already live online	Needs a connection; can break if the site changes

For anything you will keep, the folder is the best default. The Markdown stays short, the images are ordinary files you can open, replace or compress, and the whole folder travels together. Chapter 5 covers working with files on disk, and Chapter 6 covers what happens to images when you export.

Tables

Tables are a GitHub Flavored Markdown extension, and of everything in this chapter they look the most different in source and in the result. Pipes separate the columns, and a row of dashes

separates the header from the body:

```
| Option           | Cost   | Ready by |
| ---------------- | -----: | :------: |
| Stay and refit   | 1.2 M  | June     |
| Harbour site     | 1.8 M  | March    |
```

This renders as a three-column grid with a header row. The colons in the separator row set the alignment. `---:` right-aligns a column, which suits numbers, `:---:` centres it, and `:---` or plain `---` aligns it to the left.

The pipes do not have to line up. This is the same table:

```
| Option | Cost | Ready by |
| --- | ---: | :---: |
| Stay and refit | 1.2 M | June |
| Harbour site | 1.8 M | March |
```

Aligned columns are easier to read in the Markdown pane, and ragged ones need no re-padding when a cell grows. If a cell needs a literal pipe character, write it as `\|`.

In the visual pane, **Insert table** in the format toolbar puts a three by three table at the cursor for you to fill in.

When a table helps

Use a table when the reader will compare across rows or down columns, such as options against criteria or people against tasks. If each row is really a short paragraph, use a list or a short section per item instead.

Markdown tables are deliberately simple. Each cell holds one line of inline content. Emphasis, links and code work, but there are no lists, multiple paragraphs, merged cells or column widths. Keep tables to three to five columns with short cells. If a table needs sideways scrolling in the Markdown pane, nobody will want to keep it up to date.

Code blocks

For more than a fragment of code, use a *fenced* code block: a line of three backticks, the code, and another line of three backticks. Put a language name right after the opening fence as a hint:

````
```python
def total(costs):
    return sum(costs)
```
````

Everything between the fences is shown exactly as written, in a monospaced font, with line breaks and indentation kept and no Markdown formatting applied. MD Studio's Markdown pane uses the language hint to colour the code, and other tools, GitHub and static site generators among them, use it to colour the code for readers. Common hints are `python`, `javascript`, `json`, `bash`, `sql`, `yaml` and `markdown`. Leave the hint off, or write `text`, for output, logs or anything else that is not a

programming language.

Three tildes (~~~) work as a fence too, which helps when the code itself contains a line of three backticks, as it does when you write about Markdown. A longer fence of four backticks also works. That is how the example above shows a fence inside a fence.

In the visual pane, **Code block** in the toolbar turns the current paragraph into a code block. The visual editor is also set up to start one when you type three backticks and a space at the start of a line. While the cursor is in a code block, the **Text style** menu shows a dash, since a code block is neither a paragraph nor a heading.

Lines indented by four spaces also make a code block, the older *indented* form. It cannot carry a language hint and is easy to trigger by accident, so use fences.

Block quotes

Start a line with `>` to quote it:

```
> We want everyone within a 30-minute commute.
>
> From the staff survey, 2026
```

The visual pane sets the quote off with a bar and indentation. A line holding only `>` keeps two paragraphs inside the same quote. Quotes can hold any other Markdown, lists and headings included, and they nest: `>>` is a quote within a quote, handy when you reply to an email that quoted someone else.

Use quotes for words that are not yours, such as a source, a requirement or a customer's message. In the visual pane, **Quote** in the format toolbar wraps the current paragraph in a quote, and the visual editor is set up to do the same when you type `>` at the start of a line.

Callouts

Obsidian's callouts are block quotes with a label on the first line, for a note or a warning that should stand out:

```
> [!warning] Check the date
> The deadline moved to Friday.
```

The label is the type in square brackets after a bang, with an optional title after it. Without a title, the type's name is the title. Read view draws a callout with a coloured frame and hides the bracket label. The other views show the quote and the label as written, and every other tool shows a plain quote. Common types are `note`, `tip`, `warning`, `danger`, `success`, `question`, `example`, `quote` and `abstract`, and a few aliases such as `info`, `error` and `summary` map onto them.

Footnotes

A footnote moves a remark or a source out of the sentence. Mark the spot with a caret label in

square brackets, and write the note anywhere in the document with the same label and a colon:

```
The harbour site floods about once a decade.[^flood]

[^flood]: Municipal climate report, 2025, page 14.
```

The label can be a number or a word. Words are easier to keep straight when you move paragraphs around. The visual pane renders the marker as a small reference and the note as a footnote definition.

Footnotes are less portable than anything else in this chapter. GitHub renders them, and so do MD Studio's visual pane and many other tools, but they are not part of CommonMark, and a tool that doesn't know them shows the raw `[^1]` markers. MD Studio's exports number them for you: each marker becomes `[1]`, `[2]` and so on, in the order they appear, and the notes follow as a numbered list at the end. For important sources, an inline link or a *Sources* section may still serve better.

Math and diagrams

Put a formula between dollar signs and the visual pane typesets it: `$E = mc^2$` inline, or a block between two lines of `$$` for a display formula. The text between the signs is TeX. Math follows the usual rule for dollar signs, so prices stay prices: an opening `$` is not followed by a space, a closing one is not preceded by a space or followed by a digit, and a backslash before a dollar sign makes it plain text. Code never holds math. A formula is one unit in the visual pane, like an embed: to change it, edit the Markdown pane. A web page export carries formulas as MathML, so any browser typesets them; a Word document gets each formula as a picture, and a PDF gets display formulas as pictures while inline formulas stay as their TeX text. A formula that cannot be drawn keeps its TeX text. A note shown inside another with `![[Note]]` typesets its formulas too.

A fenced code block whose language is `mermaid` is a diagram:

````
```mermaid
graph TD
  A[Start] --> B[Done]
```
````

Read view draws it, and a click on the drawing shows it full size. The other views show the code. The diagram engine (Mermaid) is large and loads only when a document has a diagram, so the first one takes a moment, and in a browser you have never opened a diagram in, it needs a connection once. A diagram that cannot be drawn leaves its source on the page with a note. Exports draw the diagram too: as a picture in the web page, the Word document and the PDF. Callouts and diagrams in a note shown inside another note are drawn as well.

Horizontal rules

Three or more dashes, asterisks or underscores on a line of their own make a horizontal rule, a line across the page:

```
---
```

Use one to mark a shift that does not call for a heading, for example between a summary and the details, or between two unrelated notes in a log. **Horizontal rule** in the format toolbar inserts one.

Watch out for one thing: a line of dashes directly under a line of text makes a level-2 setext heading (see Chapter 3). Leave a blank line above `---`, or use `***`, which has no second meaning.

CommonMark, GitHub Flavored Markdown, and what to avoid

Markdown began as a loose description, and for years every tool read it a little differently. **CommonMark** is the precise specification that settled the core: paragraphs, headings, emphasis, links, images, lists, quotes, code and rules. **GitHub Flavored Markdown** (GFM) is CommonMark plus a few extensions: tables, task lists, strikethrough and bare-address autolinks.

MD Studio reads CommonMark plus GFM, and its visual pane also renders footnotes. Most tools agree on that set, so a document written in MD Studio reads the same on GitHub, in most note apps and in most static site generators. Appendix B lists every construct with its source and result.

Markdown also allows raw HTML in the middle of the text. Avoid it. It is unreadable as plain text, it behaves differently from tool to tool, and MD Studio does not render it. HTML exports show it as text instead of running it, so a document someone sends you can never run code in the exported page. If you find yourself reaching for HTML to get a layout, the plain Markdown version is usually the better document anyway.

Now you try

Take the document you structured in the last chapter and add to it.

1. Make the one phrase a skimming reader must not miss **bold**, and nothing else. Cross out a detail that has changed with `~~strikethrough~~`.
2. Select a word in the visual pane, click **Link**, and give it an address. Then clear the address to remove the link again.
3. Paste a screenshot. If your document is saved on disk in Edge or Chrome, choose its folder and look for the new `images` folder beside it. Replace the alt text with a real description.
4. Compare two or three options in a small table, and right-align the column of numbers with `---:`.
5. Add a fenced code block with a language hint, even if it is only a few lines of `json`, and look at the colours in the Markdown pane.
6. Quote a sentence from a source, and put the source itself in a footnote.

Then read the result in the Markdown pane alone (**Ctrl+2** in the installed app, or **Markdown** in the toolbar). If it still reads like a well-kept text file, the formatting is doing its job.

The next chapter is about where your documents live: files, folders, and keeping them safe on your own machine.

Managing Files Locally

File access on Windows, local persistence, working across devices

Most writing apps answer the question "where is my document?" with "in our cloud". MD Studio answers it with "wherever you put it". There is no account to sign into and no server holding a copy. Your document is a `.md` file on your own disk, and the browser keeps a second copy as a safety net while you work.

That is simple, but it is not what most people are used to. This chapter covers where your words live, what Save does in each browser, what the browser remembers for you, and how to carry work between machines without a sync service.

Two places your document lives

A document you are editing in MD Studio can exist in two places at once. The first is the file on disk, a real `.md` file in a folder you chose: Documents, a project folder, a synced OneDrive folder. That is the durable copy, the one you back up, commit to git, attach to an email or open in another editor next year.

The second is the browser's copy. Every open document, saved or not, is also kept in the browser's local storage, which is how your tabs come back when you close the window and open MD Studio again tomorrow.

The unsaved-changes dot next to the file name marks the gap between the two. With no dot, the file on disk matches what you see. With a dot, some of your edits exist only in the browser so far. Press **Ctrl+S** and the dot goes away.

The browser copy protects you from accidents in the next five minutes. The file protects you from everything else.

Saving in Edge and Chrome

Edge and Chrome (and other Chromium-based browsers on the desktop) support the File System Access API. That is a browser feature that lets a web app read and write a file you have explicitly handed to it. MD Studio uses it so that saving works the way it does in a desktop editor.

Here is the everyday loop:

1. Open a file with **Open file…** on the start screen or in the menu, or **Ctrl+O**. The system's file picker appears; choose `notes.md`.
2. Edit. The unsaved-changes dot appears.
3. Press **Ctrl+S** (or **Save** in the toolbar). MD Studio writes straight back to `notes.md`, in place, and a toast confirms: "Saved notes.md".

Nothing is downloaded, and no `notes (1).md` piles up in your Downloads folder.

A brand-new document has no file yet, so its first **Save** asks where to put it and what to call it. After

that it behaves like any opened file. **Save as…** (**Ctrl+Shift+S**) always asks, which is how you branch a copy: save `proposal.md` as `proposal-v2.md`, and from then on Save writes to the new file, leaving the original untouched.

The link to the file survives closing the app. Close the window, come back tomorrow, press **Ctrl+S**, and the save still goes to the same `notes.md`. The browser may ask once more whether MD Studio may edit the file. That is expected: browsers check file permission again after a restart, and one click restores it.

What you are granting

When the browser asks whether MD Studio may edit `notes.md`, the answer covers that one file. MD Studio cannot browse your disk, list your folders, or open other files on its own. It only sees what you pick in a file picker or drop onto the window.

If you say no, nothing breaks. Save simply falls back to asking for a location, as if the document were new.

Three ways in, one difference

You can bring a file into MD Studio in three ways, and they differ in one detail that matters for saving. **Open file…** (or **Ctrl+O**) links the document to the file, so Save writes back. Double-clicking a `.md` file on Windows does the same once MD Studio is installed as an app; the first time, right-click the file, choose *Open with* and pick MD Studio.

Dropping a file onto the window is different. It opens the contents, but browsers do not hand over a writable link for dropped files, so the first Save asks where to put it. Drag and drop is fine for a quick read. For a file you mean to keep editing, use **Open file…** or double-click it.

Saving in Firefox, Safari and on phones

Firefox, Safari and mobile browsers do not offer the File System Access API. MD Studio still works there, but files move in and out differently. Opening uses the standard file picker: you pick a file, MD Studio reads its contents, and its connection to the file ends there. Saving downloads the document. The toast says "Downloaded notes.md" instead of "Saved notes.md", and the file lands wherever your browser puts downloads.

So you manage the files by hand. If you opened `Documents/notes.md` and saved, the new version is in Downloads, and your browser may have named it `notes (1).md` to avoid a clash. Move it back over the original when you are done, or set your browser to ask where to save each download, which gets you close to the desktop experience.

The unsaved-changes dot clears after a download, because the content has left the app. MD Studio cannot tell whether you put the file back where it belongs.

The Recent list

The start screen's **Recent** list holds the files you have opened or saved, most recent first. Each

entry carries a label that tells you what clicking it will do.

An entry marked **File on disk** (Edge and Chrome only) still has its link to the file. Clicking it reopens the file itself and reads its current content from disk, so you see changes made in another editor, and Save writes back to it. An entry marked **Saved copy** is only a snapshot of the content as it was when you last opened or saved it. That is what you get in other browsers, and for files that were dropped rather than opened.

When a **File on disk** entry cannot be reached, MD Studio falls back to the snapshot and tells you why. If the file was moved or deleted, you see "Couldn’t find notes.md on disk any more, so opened the saved copy." If you declined the browser's permission prompt, you see "Opened the saved copy of notes.md. Allow file access to work on the file itself." Either way you get your words back, but you are no longer connected to the file.

The list keeps your most recent handful of files. Very large documents are left out of it, to protect the browser's storage. The **×** beside an entry removes it, and **Clear list** removes them all after asking "Clear the recent list?". As the dialog says, the files themselves are not touched; they just stop being listed.

The safety net: browser storage

MD Studio keeps everything you have open in the browser's local storage, without being asked. Every tab and every unsaved change is written a fraction of a second after you stop typing, and again the moment you switch away from the window.

If you close the browser tab by accident, reopening MD Studio brings it all back: the start screen offers **Continue editing** with your document's name, and your tabs are where you left them. A browser crash or a Windows restart for updates ends the same way, with at most the last moment of typing missing. When an update to MD Studio arrives, a toast says "A new version of MD Studio is ready." with a **Reload** button, and reloading is safe because your open documents are already stored.

That comfort is also where people get into trouble, because browser storage has limits a file on disk does not. It belongs to one browser on one device: your Edge at work and your Edge at home do not share it, and neither do Edge and Chrome on the same machine. It can be cleared by "Clear browsing data" with site data selected, by a privacy cleaner, by a reset browser profile, or by a managed work laptop that wipes data on sign-out, and none of them ask MD Studio first. An InPrivate or Incognito window throws its storage away when it closes. And each site gets only a few megabytes; plain text fits easily, a document stuffed with embedded images may not.

Treat the browser copy as an airbag. It is there for accidents. Saving to a file is what makes your work durable.

What "local-first" means here

"Local-first" gets stretched to mean many things. In MD Studio it means you never sign in, because there is no account. Your documents are never uploaded: the website delivers the app to your

browser and then stays out of the way. MD Studio does not copy your documents between devices either, and it could not, since nothing of yours leaves the device.

It also means the app works offline once it has loaded. The first time it is ready, a toast says "MD Studio is ready to work offline." Install it (see Chapter 1) and it opens like any other program, network or not.

What you get is privacy, speed and files you own outright. In return, you decide where those files live.

Images and the folder next to your document

Images complicate "a document is a file", because a picture is a second file. MD Studio handles this in one of two ways, depending on whether it can reach the folder your document sits in. (Chapter 4 covers images as content; this is the file side.)

In Edge and Chrome, with the document saved on disk, paste an image (**Ctrl+V**) or drop one into either pane, and the first time MD Studio asks "Save images next to the document?". Choose **Choose folder…**, pick the folder that contains your document, and from then on images go into an `images` folder beside it:

```
Reports/
 quarterly-review.md
 images/
     image-20261002-091544.png
     image-20261002-093012.png
```

The document links to them with a short relative path such as `images/image-20261002-091544.png`. Your Markdown stays small and readable, and the pictures travel with the folder: copy the folder, sync it, or commit it to git, and everything still lines up. If you pick the wrong folder, MD Studio says so ("That folder doesn't contain quarterly-review.md. Choose the folder the document is in.") and embeds that one image instead; the next paste asks again.

In other browsers, before the document has been saved, or if you choose **Cancel** in that dialog, the image is shrunk to at most 1600 pixels on its longest side and embedded in the document itself. You get one self-contained file with no folder to keep track of. The cost is size: a few screenshots can turn a 10 KB text file into a 2 MB one, and the source pane fills with a long line of encoded data. When an embedded image makes the document large, MD Studio warns you and suggests saving to disk so images can go in a folder instead.

So save a new document to disk *before* you start pasting images into it, and your Markdown stays clean.

The attachment folder and Obsidian vaults

If your notes come from Obsidian, or you keep pictures and PDFs in a folder tree of their own, MD Studio can show them where the Markdown names them. This works in Edge and Chrome, with the

document saved on disk. The idea is one folder per document that holds the note and everything it points to. Choose the vault's top folder, the one with the `.obsidian` folder in it, because a name like `![[plan.png]]` can refer to a file anywhere in the vault, and MD Studio can only look inside the folder you chose.

When you open a note that has embeds the files cannot be found for, a bar appears above the editor. It says "Some attachments in this note cannot be shown" and has a **Choose attachment folder** button. Click it and pick the folder. The command palette has the same command, **Choose attachment folder…**. The folder must contain the note; if it does not, MD Studio tells you and keeps the old choice. Until then each embed is a labelled placeholder with its original text, so you can still read and edit the note.

The bar has other forms. After you reload the page or restart the browser, it may say that attachments in the folder "need permission before they can be shown", with an **Allow access** button; the browser wants one click from you before it lets a page read a folder again. If the document has not been saved, it says "Save this document to disk to show its attachments." In a browser that cannot read folders, it says so and the placeholders stay. And if you chose a folder inside the vault, so that some files cannot be found, it says "Some attachments were not found" in that folder and asks you to choose the vault root. A missing file's placeholder also carries a tooltip with the same advice. The **×** closes the bar for that tab. Embeds that appear only inside code, such as in a guide that explains the syntax, do not raise it, and neither do embeds of a section of the same note or of a web address, which need no folder. The "not found" count follows each embed as it is drawn: fix a typo in an embed and the bar can stay until you run **Rescan attachments** or reload. The palette commands tell you when they cannot run (browser unsupported, document not saved, no folder chosen yet, access to allow) instead of reporting a rescan that did not happen.

The first time a large vault is opened, MD Studio indexes it in the background. You can start reading at once: a bare file name is tried in the note's folder, the vault's attachment folder and the vault root first, so most pictures show before the index is done. While the index is incomplete a file that cannot be found yet shows "Loading…" instead of "Not found". The index holds up to 50,000 files; a bigger vault gets a notice, once per session, and some files may then show as not found. A paste or drop that happens before the index is complete waits at most two seconds for it; if it is still incomplete, the link uses the full path instead of a bare name, so it cannot point at the wrong file. MD Studio has no file watcher, so a file you add in another program shows after you return to the window, after a missing one is looked for again, or when you run **Rescan attachments** from the command palette.

MD Studio reads three settings from `.obsidian/app.json` in the vault: the attachment folder, whether new links are Markdown or wikilinks, and whether paths are the shortest, relative or absolute. It never changes that file, and if it cannot read it (a permission or disk error, as opposed to a missing file) MD Studio refuses to save the attachment and says so, rather than guess a folder. A vault with a `.obsidian` folder but no `app.json` gets Obsidian's defaults, which are the vault root, wikilinks and the shortest path. With the relative path style, a wikilink to a file with no folder in its name is written as `![[./photo.png]]` when the note is not in the vault root, so it cannot match a root file of the same name. A dropped picture whose name has no image extension (`photo`, or a browser's `.jfif`) gets the right one (`photo.jpg`). A folder with no `.obsidian` folder keeps the older behaviour: images in an `images` folder beside the document, linked the Markdown way.

The folder choice belongs to the document. If you use Save As to move a note to another place inside the same folder tree, the folder follows it. If the new location is outside the folder, MD Studio asks again when it is needed. A document with a folder already chosen for another note in the same vault picks up that folder without asking, preferring the vault's top folder when it has several to choose from.

A folder of notes

When your notes live in a folder, open the folder itself: **Open folder** on the start screen, in the command palette, or the **Folder** button in the toolbar. In Edge and Chrome, pick the folder once and its Markdown files appear in a panel beside the editors, folders first, with a box at the top to filter them by name. A click opens a note in a tab, or goes to the tab that already holds it. The note you are reading is highlighted.

Below the list, **Linked from** names the notes in the folder that link to the open one, whether by a wikilink such as `[[Plan]]` or a Markdown link such as `[plan](Plan.md)`. Links inside code do not count. MD Studio reads the notes to find them, up to the first 1,500, and remembers what it read until a note changes.

A note opened from the folder uses the folder as its attachment folder, so its pictures and links to other notes resolve with no further question. In Read view a click on such a link opens the other note. The **Back** and **Forward** buttons beside the file name, or Alt+Left and Alt+Right, step through the notes you opened by following links or from the panel, like a browser's history. On a phone or tablet in Read view, swipe right to go back and left to go forward.

A link to a file that is not a note, such as `[report](report.pdf)` or `[[report.pdf]]`, works the same way: a PDF, picture, recording or video opens in a new tab, and anything else is saved to your downloads. SVG and HTML files are always saved rather than opened, because a page opened from the folder would run with the app's own access.

The panel is a tree you can drive from the keyboard: Up and Down move between rows, Right opens a folder and then steps into it, Left closes it and then steps out, Home and End jump to the ends, and Enter opens a note. On a narrow screen the panel slides over the page, and picking a note puts it away.

The folder is remembered. After you restart the browser the panel shows its name and a button, **Allow access**, because the browser asks for one click before it lets a page read a folder again. **×** in the panel's header forgets the folder.

Working across devices

MD Studio does not sync, so moving between a work laptop, a home desktop and a phone comes down to where you keep the files. You probably have the right tool already.

A synced folder

Keep your `.md` files in a folder that OneDrive, Dropbox, iCloud Drive or Google Drive already synchronises. Then:

1. On the work laptop, open `OneDrive/Writing/plan.md` in MD Studio, write, and press **Ctrl+S**. The file is saved in place, and OneDrive uploads it in the background.
2. At home, open the same file from the same synced folder. You are looking at the version you saved at work.

MD Studio does the editing and the sync client does the moving, and neither needs to know about the other. Images saved into the `images` folder ride along, because they live in the same synced folder.

Two habits keep this smooth. First, save before you leave a machine: unsaved changes live only in that browser's storage and will not follow you home.

Second, let MD Studio notice. When you come back to the window, or switch to a tab, MD Studio checks whether the file changed on disk since it last read or wrote it. A tab with no unsaved changes quietly picks up the newer version. A tab with unsaved changes keeps your work and tells you the file changed, and **Save** then asks before writing over the newer version. Answer **Cancel** if you want to compare first: open the file again in another tab and copy across what you need.

A git repository

If you write documentation, or simply want a full history of every draft, keep your files in a git repository. MD Studio saves the `.md` file and the `images` folder in place, and you commit them with whatever git tool you already use. Git was built for plain text, and every change shows up as a readable line-by-line diff. Chapter 7 says more about Markdown and version control.

Phones and tablets

On a phone, the system file picker can usually reach iCloud Drive, OneDrive, Dropbox or Google Drive. Save downloads a copy rather than writing in place, so you move the saved file back yourself. That works for reading and quick fixes. For long sessions, a desktop with Edge or Chrome is smoother.

Recovering from a closed tab or a crash

Roughly in order of how often they happen, these are the usual mishaps and what to do about each.

If you closed the browser tab or window, open MD Studio again. Your documents are still there, with any unsaved changes, under **Continue editing** and in their tabs. Press **Ctrl+S** to get them onto disk.

If you closed an MD Studio tab inside the app and it had unsaved changes, MD Studio asked first ("Discard unsaved changes?"). If you clicked **Discard**, those edits are gone. If the tab was saved, reopen it from **Recent**.

A browser crash or a restart is handled like a closed window: reopen MD Studio. At most the last moment of typing is missing.

If browser data was cleared, the open tabs and the Recent list are gone, because they lived in that

storage. Your files on disk are untouched; open them again with **Open file…**. This is the case that makes regular saving worth the keystroke.

If you saved over something you wanted, MD Studio cannot help much, because it keeps no version history of its own. Undo (**Ctrl+Z**) works for as long as the document is open in that pane. Beyond that, your safety lies outside the app: OneDrive's and Dropbox's version history, Windows File History, Time Machine on a Mac, or git. If the document matters, make sure one of those is running.

Now you try

Ten minutes is enough to see this chapter in action.

1. In Edge or Chrome, create a new document and press **Ctrl+S**. Save it as `practice.md` in a folder your cloud service syncs. Note the toast: "Saved practice.md".
2. Type a sentence. Watch the unsaved-changes dot appear. Close the whole browser window without saving.
3. Open MD Studio again. Your sentence is there, the dot is still there. Press **Ctrl+S**; the browser may ask for permission to edit the file; allow it.
4. Open `practice.md` in Notepad or any other editor. Your sentence is in the file.
5. Paste a screenshot into the document. Choose **Choose folder…**, pick the folder, and look in it: a new `images` folder with your screenshot inside. If the folder is an Obsidian vault, the file lands where the vault's settings say and is linked as `![[Pasted image …]]` instead.
6. Go to the start screen and find `practice.md` under **Recent**, labelled **File on disk**.

If you have a second computer, open the same file there from the synced folder. That is all cross-device work in MD Studio involves: a file in a folder you already trust.

The next chapter takes the same file outward, into formats for people who do not use Markdown at all: Chapter 6.

Exporting and Sharing

Export formats, and what travels well and what doesn't

Sooner or later, a document has to leave your desk. A colleague needs the proposal, a client wants a PDF, the meeting notes belong in an email. MD Studio gives you several ways out, and choosing well is mostly a question of who is on the other end and what they will do with the document.

This chapter starts with the format people overlook, the Markdown file itself. Then it goes through each export in turn, and ends with what survives the trip and what gets lost.

Share the `.md` file itself

Before you export anything, ask whether the recipient could simply take the `.md` file. Often they can, and then it is the best option you have.

A Markdown file is plain text. Anyone can open it in Notepad, TextEdit, VS Code, or MD Studio itself, and it reads cleanly even unrendered: a `#` in front of a title and asterisks around a word do not get in anyone's way. GitHub, GitLab, Azure DevOps, Obsidian, Notion and many wikis render it on sight. It is also the only format the recipient can edit and hand back to you with nothing lost in conversion.

Send the `.md` when the reader is technical or works in a tool that renders Markdown, when you want edits back rather than comments, or when the document is headed for a repository or a wiki. It is also the right choice for a document with a long life ahead, because you will not have to keep two versions in step.

Check the images before you attach it. If they were saved to an `images` folder next to the document (see Chapter 5), the `.md` file only holds links to them, so send the folder, zipped, along with the file. In a git repository or a shared OneDrive folder this takes care of itself. If the images are embedded instead, the `.md` is self-contained, but it may be large, and the long lines of encoded image data make the source harder to read.

The Export dialog

When the recipient needs something other than Markdown, open the **Export** dialog: **Export…** in the menu, or *Export…* in the command palette (**Ctrl+K**). The dialog says what it does in one line: "Save a copy in another format. Your Markdown file stays as it is."

Take that literally. Export writes a *separate* file. It never changes which file your document is linked to, so after exporting `report.pdf`, **Ctrl+S** still saves `report.md`, not the PDF. You can export as often as you like without any risk to your source.

Pick a **Format**:

Format	What the dialog promises
Web page (HTML)	Standalone page styled like the visual view.

PDF	A4 document for sharing and printing.
Word document (DOCX)	Editable in Word, with real headings, lists and tables.
Plain text	Formatting marks removed, structure kept.

For HTML, a **Page theme** setting also appears. Then click **Export**.

In Edge and Chrome, a save dialog asks where to put the file, suggesting a name built from your document's: `report.md` becomes `report.pdf`. Elsewhere, the file downloads. A toast confirms either way: "Exported report.pdf" or "Downloaded report.pdf".

The dialog remembers the format and theme you used last, so if you export to PDF every Friday, it opens on PDF. If you always export the same way, skip the dialog: the command palette has one command per format, *Export as HTML*, *Export as PDF*, *Export as Word document* and *Export as plain text*, each using your last-used page theme.

Web page (HTML)

The HTML export is a single, self-contained `.html` file styled like MD Studio's visual view: comfortable line length, clear headings, readable tables and code. It opens in any browser on any device, with nothing installed.

Page theme has three choices. **Follow reader's system** shows the page light to a reader in light mode and dark to one in dark mode, which suits anything read on screen. **Light** and **Dark** fix the look; pick **Light** for anything that may be printed or projected.

HTML is the export that travels best. It keeps everything Markdown expresses as structure: headings, nested lists, task lists, tables, block quotes, code, links. It is also the easiest to paste from (see below), and when printed from a browser, the page drops its background and prints black on white.

Images come along in three ways. A picture pasted into the document (PNG, JPEG, GIF or WebP) is embedded in the page itself, and so is a picture read from the folder you chose for the document, an Obsidian attachment included, at the width the note gives it. Images linked by a relative path that MD Studio cannot read stay as links, so they appear when the `.html` file sits next to the `images` folder. A picture over 5 MB, or any picture past 25 MB in total, is left out of the page to keep it a sensible size; the description is shown in its place and MD Studio says how many were left out. Word and PDF keep them, and WebP, AVIF, BMP and SVG pictures go into the page as they are, without being redrawn. Images on the web, including a web address written as an embed, `![[https://example.com/a.png]]`, follow **Load web images** in Settings, as they do in the visual pane: with it off, which is the default, the page shows the image's description in a dashed frame instead of loading it, so the exported file never makes the reader's browser contact that server. Turn it on and the page links to them, and they work while the reader is online.

Raw HTML in your Markdown is shown as text rather than run, and a link that could run a script loses its address but keeps its text. Ordinary web, email and relative links stay clickable. The reason is simple: a page exported from a document someone sent you should never run code from that document.

PDF

The PDF export produces an A4 document, set in clean standard fonts, with headings, lists, tables, quotes and code blocks laid out on the page. Nested lists keep their indentation, and task list items show `[x]` or `[ ]`. It is the format for "please send me something I can read on a plane", for formal attachments, and for anything that must look the same everywhere.

Know its main limit in advance. To stay small and fast, the PDF uses the standard fonts every PDF reader has built in. Those cover Latin text, including Danish **æ**, **ø** and **å**, accented letters and ordinary punctuation. They do not cover emoji, Greek, Cyrillic, Chinese, Arabic and other scripts, and those characters are left out of the PDF rather than breaking the export. Images come along when MD Studio can read them: pictures pasted into the document, and pictures in the document's attachment or `images` folder when the document is saved on disk in Edge or Chrome. PNG and JPEG are embedded, and WebP, AVIF, BMP and SVG are redrawn as PNG first, at the size a note asks for with `![[plan.png|300]]` or the full width otherwise. GIFs and images on the web show their alt text instead.

If your document contains any of that, use printing instead (next section): the browser's own PDF printer renders every character and every image the visual pane can show.

Word document (DOCX)

The Word export is for when the next person will keep working on the document in Word, or a template, a review process or a client requires a `.docx`.

The result uses real Word structure. Your `#` headings become Word's built-in Heading 1, Heading 2 and so on, so the navigation pane, a generated table of contents and the document's styles all work. Bulleted and numbered lists, nested ones included, become Word lists, and tables become Word tables with a header row. Links stay clickable. Code blocks are set in a monospace font, block quotes are indented, and task list items keep their state as a or in front of the text.

Images are embedded the same way as in the PDF (PNG, JPEG and GIF, with WebP, AVIF, BMP and SVG redrawn as PNG), and anything MD Studio can't read becomes a short placeholder with its alt text. Once the `.docx` leaves MD Studio, it is a Word document. Edits made there will not flow back into your Markdown, so decide which copy is the master before the review starts.

Plain text

The plain text export removes the formatting marks and keeps the structure, for destinations that accept nothing else: a ticketing system with a bare text box, an SMS, an old mainframe form, a terminal.

Top-level headings are underlined with `=` and second-level headings with `-`, so they still stand out. Lists keep their bullets (`-`) and numbers, and task items show `[x]` or `[ ]`. Tables become rows of tab-separated cells, which paste straight into a spreadsheet. A link keeps its address in brackets after the text, like `the report (https://example.com/report)`. Code is kept exactly as written.

The `.txt` file looks a lot like your Markdown without the asterisks and backticks, which is no accident, since Markdown was designed to read well as plain text.

Printing, and the other road to PDF

Ctrl+P, or *Print* in the command palette, prints the formatted document, after every embedded picture, PDF and note has loaded. The toolbar and the Markdown pane are left out, so the printout is the formatted document, not the source. Print from **Split** or **Visual** view, so the formatted pane is up to date with your latest edits.

The browser's print dialog has a *Save as PDF* (or *Microsoft Print to PDF*) destination, and that gives you a second road to PDF with different strengths:

	Export as PDF	**Print to PDF**
Fonts	Standard fonts; Latin text only	Your system's fonts; every script and emoji
Images	Left out	Included, as shown in the visual pane
Page size	A4	Whatever you choose in the print dialog
Margins, headers	Fixed	Set in the print dialog

Use **Export** for a quick, consistent A4 document. Use **Print** when the document has images, emoji or non-Latin text, or when you need US Letter or custom margins.

Exporting offline

MD Studio works offline, and so do HTML, plain text and printing. PDF and Word exports rely on larger components that MD Studio downloads only the first time you use them, so the first PDF or Word export needs a connection. After that, they work offline too. If you know you will be exporting on a train, run one PDF and one Word export at your desk first.

What survives the trip

No conversion is perfect. The table shows what to expect from each route, so you find the gaps at your desk rather than in the recipient's inbox.

In your document	**.md**	**HTML**	**PDF**	**Word**	**Plain text**
Headings, lists, quotes	Yes	Yes	Yes	Yes	As text structure
Tables	Yes	Yes	Yes	Yes	Tab-separated rows
Links	Yes	Clickable	Shown as text	Clickable	Address in brackets

Task lists	Yes	Checkboxes	[ ] and [x]	and	[ ] and [x]
Strikethrough	Yes	Yes	Yes	Yes	Plain text
Pasted images	Yes, inside the file	Yes	PNG, JPEG, WebP	PNG, JPEG, GIF, WebP	Alt text
Images in an images or attachment folder	If you send the folder	Embedded when saved on disk in Edge or Chrome	PNG, JPEG, WebP, AVIF, BMP, SVG, when saved on disk	PNG, JPEG, GIF, WebP, AVIF, BMP, SVG, when saved on disk	Alt text
Obsidian ![[...]] embeds of PDF, audio, video, other files	Yes	A labelled line	A labelled line	A labelled line	A labelled line
Obsidian note embeds	Yes	Written out as a quote	Written out as a quote	Written out as a quote	Written out in place
Footnotes	Yes	Numbered notes at the end	Numbered notes at the end	Numbered notes at the end	Numbered notes at the end
Raw HTML	Yes	Shown as text	Mostly left out	Shown as code	Kept as text
Emoji, non-Latin scripts	Yes	Yes	Left out	Yes	Yes

Headings, tables and simple lists come through every format, which is a good reason to carry meaning in structure rather than in visual tricks.

Images are the format that needs the most care. Images on the web are never fetched for a PDF or Word export, so link to a copy in the `images` folder if the picture has to travel. In an HTML export they are linked only when **Load web images** is on. And remember that a relative link such as `images/chart.png` means "next to this file": move the file without its folder and the link points at nothing.

Footnotes are not part of CommonMark, so the exports turn them into something every format understands: each marker becomes a number in brackets, and the notes follow as a numbered list after a rule at the end. Raw HTML is handled differently by each format, as the table shows. Appendix B marks which syntax is CommonMark and which is GitHub Flavored Markdown. If a document has to export cleanly, stay within those two.

Embeds and wikilinks in exports

An export is made from a copy of the text, so your note stays as it is. The copy is changed in a few ways to suit a file that cannot go and fetch things.

A picture embed, `![[plan.png|300]]` or an ordinary Markdown link to a file in the folder you chose, becomes a picture at the width you wrote, in HTML, Word and PDF. A PDF, audio, video or other file embed cannot play inside a static document, so it becomes a line naming it, such as

"PDF: report.pdf". A plain wikilink `[[Meeting notes|the notes]]` becomes its alias or its name as ordinary text.

A note embed is written out in place as a quote, to a depth of three, and a note that would repeat itself is left out with a short notice. That works also inside a list item or a quote, which keeps its indentation. When the embed shares its paragraph with other text, or sits in a table cell, there is no clean place for a quote, and it becomes a one-line label with the note's name instead. The embedded notes of one export add up to at most about a million characters; past that the rest are left out with a notice saying the embedded notes add up to too much text. A note's own reference links and footnotes stay with it, and the Markdown links in your note whose label is a bracketed name, such as `[[1]](https://example.com)`, stay links.

Anything that cannot be found, or whose folder cannot be read, becomes plain text such as "Attachment not found: plan.png", "Note not found: Meeting notes" or "Attachment not included: plan.png", never a blank or a broken picture. If the folder needs permission again, for example after a reload, MD Studio shows a notice and exports with those labels; allow access from the bar above the editor and export again to include them.

Pasting into email, chat and wikis

Often you do not want a file at all. You want the text somewhere else, now.

For email, Teams, Word or Google Docs, select in the visual pane and copy. You get formatted text with headings, bold, lists and links intact, much as it looks in the visual view.

For a destination that understands Markdown itself, such as GitHub issues and pull requests, GitLab, Azure DevOps or Obsidian, select in the Markdown pane and copy. You get the raw source, and the destination renders it.

For a long or important email, export to HTML with the **Light** theme, open the file in your browser, select all (**Ctrl+A**), copy, and paste into the email. That gives the most predictable formatting, tables included.

And when the destination mangles everything you paste, export to plain text or copy from the Markdown pane. Markdown holds up well with no formatting at all.

Now you try

Take a document with a heading, a list, a table and a link, ideally one you need to send this week.

1. Open **Export** and export it as **Web page (HTML)** with **Follow reader's system**. Open the file in your browser, then switch your system between light and dark mode and watch the page follow.
2. Export it again as **PDF**. Notice the dialog opened on HTML, the format you used last; pick PDF this time. Open the PDF and check the table.
3. Press **Ctrl+P** and choose *Save as PDF*. Compare the two PDFs side by side: fonts, margins, page size.
4. Export as **Word document (DOCX)**, open it in Word, and open the navigation pane: your

   headings are there, as real headings.
5. Finally, select a paragraph and a list in the visual pane, copy, and paste it into a new email. Then do the same from the Markdown pane, and compare.

Then press **Ctrl+S** and look at the file name in the toolbar. It is still your `.md` file; four exports later, the source has not changed. Chapter 7 steps back from the buttons to the writing habits behind them.

Writing as Practice

Plain text as a discipline

The first six chapters were about the tool: the two panes, headings and lists, formatting, files and exports. This chapter is about the habits those features make easy. There are no new buttons in it. Every move was covered somewhere in Chapters 1 to 6, and I point back to where. What this chapter adds is the order you do things in, and the reasons for it.

I think of plain text as a way of working more than as a cheaper word processor. You keep the words in a format you own, you add structure once you know what it should be, and you let every revision leave a record you can read.

Plain text is the feature

A Markdown file is a text file with a few conventions in it. That sounds like a limitation until you think about what it means over ten or twenty years.

It stays readable. Open a `.md` file in Notepad, in a terminal, on a phone, or in an editor nobody has written yet, and you will see the words with their structure intact, because the markup was designed to be read as it is:

```
# Offsite plan

We meet on **12 March** in Aarhus.

## Agenda

1. Review of last year
2. Priorities for the next two quarters
3. Open questions
```

Nobody needs to be told that `#` starts a title or that `1.` starts a numbered list, and the asterisks around the date still read as emphasis. A word-processor file from twenty years ago is a different story. You can open it today only if the right program still exists and still agrees with how the old one stored things.

It also travels. The same file opens in MD Studio, VS Code, Obsidian, on GitHub, in a static-site generator or in Pandoc. When you write Markdown you are picking a format that any of these tools can read, and the format will outlive every app built for it, this one included.

Nobody else holds it, either. MD Studio keeps your documents on your own computer, with no account, server or sync service between you and your files (Chapter 5 covers where they live). If the app disappeared tomorrow, your files would still be ordinary files in ordinary folders. The app doesn't have to promise this, because it follows from the format, and that is my main reason for using the format in the first place.

And it is small. A year of notes is a few megabytes. Any search tool on any operating system can find a phrase in it, you can back it up by copying a folder, and a single document emails as a few

kilobytes of text.

A few habits keep this working for you. Name files so that they will still mean something in five years: `2026-offsite-plan.md` will, and `Untitled (3).md` won't. Lowercase and hyphens survive every operating system and every URL. Keep a document and its `images` folder in the same folder so they move together (Chapter 4 covers how MD Studio saves pasted images next to the document). And prefer the syntax every reader understands. Headings, paragraphs, lists, links, emphasis, tables and code blocks read well everywhere, while extended syntax such as footnotes renders in some places and not others (Chapter 6 has the details). Use the extras when you need them.

Structure: when to reach for it, and when to wait

Markdown makes structure cheap. A `#` and a space gives you a heading, and a `-` and a space gives you a list. That is useful, but it also makes it easy to add structure before you know what the structure should be.

Write first, structure second

When you don't yet know what you think, write prose. Full sentences make you connect one idea to the next with *because*, *but* and *so*. Bullets let you skip that work, and a list of five fragments can look organised while hiding the fact that you haven't decided how the fragments relate.

A first draft can be a single run of paragraphs with no headings at all. Write until you run out, then read it back and look for the seams: the places where the subject changes or a new argument starts, or where a reader would want to skip ahead. Those seams are your headings, and they fit the content because they came out of it.

Starting from a skeleton of headings and filling them in works when the shape is already known, as with a meeting agenda, a team template or a report with fixed sections. It works badly for thinking, because the headings settle the argument before you have made it.

Reach for headings when the reader needs to move

Add a heading when a reader would benefit from skipping ahead, coming back or seeing where they are. A one-page note may need none. A ten-page document needs them, or nobody can find anything in it, including you.

Where you can, make headings say something. Here are two outlines of the same document:

```
## Background
## Analysis
## Recommendation
```

```
## Support tickets doubled after the March release
## Most of the increase comes from one import screen
## Fix the import screen before adding new features
```

The first outline is a set of labels, while the second gives you the whole argument in three lines. A

reader who sees only the second already knows what you think, and you can check whether the argument holds just by reading the headings in order. You won't always manage this, and reference material and manuals are often better with plain labels. When a document argues for something, though, headings that state claims are a good test of whether there is an argument at all.

Keep the levels shallow: one title, sections under it, and rarely anything deeper than a third level (Chapter 3 covers heading levels as an outline). If you find yourself reaching for a fourth level, the section has probably become a document of its own.

Reach for lists when the content is a list

Use a list when the items really are parallel, such as the options on the table, the steps of a procedure or the people invited. Numbers are for order that matters, bullets for sets where it doesn't, and a task list (`- [ ]`) for things to do.

If the items depend on each other, a list is the wrong tool. When the third bullet only makes sense because of the second, you are writing an argument, and it wants to be a paragraph. One quick test is to ask whether you could shuffle the bullets without losing meaning. If you could, it's a set. If the order matters, number it. If shuffling breaks it because each point builds on the last, write it as prose.

The same content as bullets and as a paragraph:

```
- Import screen slow
- Users retry
- Duplicate records
- Support tickets
```

```
The import screen is slow, so users click Import again. Each click
creates another copy of the records, and the duplicates are what
people write to support about.
```

The bullets look tidier. The paragraph is the one that explains what is going on.

Plain text and version control

Because Markdown is plain text, it works with the tools programmers built for tracking changes to text, of which git is the best known. You don't need to be a programmer to get something out of git, and Markdown is about the friendliest thing you can keep in it.

MD Studio itself does not do git. It has no commit button, history panel or branches. It saves ordinary `.md` files to ordinary folders, and if one of those folders is a git repository, you run git alongside MD Studio on the files it saves. Write and save in MD Studio, then commit with whatever git tool you like: the command line, GitHub Desktop, or the git panel in another editor. The two never need to know about each other.

Why Markdown and git fit together

Git compares files line by line. When you change a Markdown file, git shows exactly which lines

changed, and you can read the change directly:

```
 ## Recommendation

-Fix the import screen before the summer release.
+Fix the import screen before adding new features. The summer
+release can wait two weeks.
```

To git, a Word document is a binary file: it can tell you the file changed, but not what changed in it. A Markdown document shows the sentence you rewrote, the paragraph you deleted and the heading you renamed, so the history becomes something you can read as well as restore from.

One sentence per line

Line-by-line comparison works best when lines are short and each one means something. Many people who keep prose in git put **one sentence per line**, or follow the looser convention called **semantic line breaks**, which adds a new line after each sentence and, if you like, at natural pauses such as a clause or a comma.

In Markdown this costs nothing on the page. A single line break inside a paragraph renders as a space, and only a blank line starts a new paragraph. So this source:

```
The import screen is slow, so users click Import again.
Each click creates another copy of the records.
The duplicates are what people write to support about.
```

shows up in the visual view as one ordinary paragraph of three sentences. Switch to the visual view and check for yourself; it makes a good first try at the two-lens habit described later in this chapter.

The benefit shows when you revise. Change one word in the second sentence and git reports one changed line instead of flagging the whole paragraph. Reordering sentences becomes moving lines, and review comments on GitHub or GitLab attach to a single sentence.

There are downsides. The source looks ragged, which some people dislike, and the habit only pays if you keep it: one paragraph written the old way among many written sentence by sentence makes the diffs uneven again. I'd treat it as an option. If you keep a document in git and revise it often, try it on that one document and decide whether the diffs are worth the ragged edge. For a one-off document I wouldn't bother.

There is one thing to know about MD Studio here. When you edit a paragraph in the visual pane, MD Studio keeps the source text of every block you didn't touch, but writes the block you edited back in the editor's own style (Chapter 2 explains why). If you care about the exact line breaks in a paragraph, edit that paragraph in the source pane, and look at the diff before you commit.

Commits as drafts

Without version control, drafts pile up as files: `report-v2.md`, `report-v2-final.md`, `report-v2-final-REALLY.md`. With git you keep one file, and each commit is a draft.

Commit when you finish a thought rather than when you finish a day. "Rewrite the recommendation

around the import screen" is a step worth returning to, and "End of Tuesday" isn't. Write the message as an editorial note, such as "Cut the background section; nobody needs it" or "Move the cost estimate ahead of the timeline", and six months later the log tells you why the document looks the way it does. Before a big change, such as a restructure, cutting a third of the text or accepting someone's heavy edit, commit what you have first.

That last habit changes how you revise. People hold on to weak paragraphs because deleting them feels permanent. When the old version is one command away, cutting gets easy, and documents get shorter and better for it.

Branches for alternatives

Sometimes you aren't sure which of two directions is right: a different opening, a reorganised middle, a shorter version for another reader. A git branch lets you try one direction without disturbing the other. Make a branch, restructure as much as you like, and compare the two versions with a diff or by reading both. Keep the one that works, merge it back and delete the other.

Most documents don't need branches. They earn their keep on long-lived documents with real alternatives, such as a proposal with two possible framings or a guide where you want to try a different chapter order. For a short note, a commit before the experiment is enough.

The minimum to get started

If you have never used git, four ideas cover it, and your git tool will have a button or command for each:

1. **Initialise** a repository in the folder where your documents live (once).
2. **Stage and commit** your changes with a message each time you finish a step.
3. **Look at the history** and the diff of any commit to see what changed.
4. **Push** to a remote such as GitHub or a private server if you want a copy off your machine or want to share.

The rest can wait. The Pro Git book in Appendix C covers it. Images MD Studio saves in the `images` folder next to a document are ordinary files, so they go into the repository with it.

Two lenses on the same thinking

MD Studio shows one document two ways, with the Markdown source on the left and the formatted document on the right (Chapter 2). It's tempting to pick one and stay there: people who like seeing the markup live in the source, and people who don't live in the visual view.

I find both views most useful as review tools. Each shows you something about your own structure that the other hides, and that goes well beyond formatting.

The outline as a skeleton check

Open the **Outline** (toolbar, or Ctrl+Shift+O). It lists the headings and nothing else, which gives you the bare skeleton of the document.

Read it from top to bottom. If the headings state claims, the outline on its own should carry the argument; if it reads like a list of labels, decide whether that suits this document or means you haven't worked out what each section says. Check that headings at the same level are the same kind of thing, because three sections named after problems and a fourth named after a person suggests something is in the wrong place. Check the levels too. A third-level heading that matters as much as the second-level ones around it should move up, and a section with a single subsection usually doesn't need the subsection. Finally, look for gaps, which are easier to spot in a skeleton than in prose. If the outline goes from problem to solution with nothing about cost, the reader will notice that too.

Do this whenever a document changes size, and always before you send it. It takes a minute, and it catches things a full reread misses, because reading the full text pulls you into the sentences.

You can get the same skeleton without the panel. In the command palette (Ctrl+K), type `#` to list the headings, then a few letters to jump to one. If you can't guess what a section is called, your reader won't be able to either.

Reading in the visual view

Switch to the visual view (the **Visual** button in the toolbar, or Ctrl+3 in the installed app) and read the document the way your reader will. Leave the editing for later.

The visual view shows you things the source hides. Formatting that didn't take, like a stray asterisk, a link whose brackets don't close or a heading with no space after its `#`, looks almost right in the source and shows up as literal symbols on the page. A list item with a missing blank line or the wrong indentation can merge into the paragraph above or split a list in two, and you'll see that at once when it's formatted. A paragraph that looks reasonable in the source can turn out to be a dense block on the page. And bold on every second phrase is easy to miss when it's just asterisks, but on the page it's obvious that nothing stands out.

Before you share a document, read it once from top to bottom in the visual view. When something makes you stop, fix it and read that part again.

Editing in the source

Switch to the source (the **Markdown** button, or Ctrl+2 in the installed app) to see what the formatting is. The visual view shows how things look, and the source shows what they are.

That matters more often than you'd expect. In the visual view, a short bold line and a heading can look alike, but only the heading appears in the outline, moves with linked scroll and becomes a heading in a Word export; the source tells you which one you have. The visual view shows a link's text, while the source shows its address and whether it is relative (`images/chart.png`, which travels with the folder) or absolute (a web URL, which may not). Indentation in the source decides how a list nests, so when a list behaves oddly, the source shows why. And the source is what you share when you send the `.md` file (Chapter 6) and what git tracks. If the visual view looks fine but the source is a mess, the mess is what other tools will see.

Some edits are easier in the source, like moving a section (select from its heading to the next one,

cut, paste), changing a heading level by adding or removing a #, or fixing a table's alignment row. Others are easier in the visual view, such as rewording a sentence while you can see the paragraph around it, or building a table from scratch. Make each edit in whichever pane suits it, and glance at the other to confirm it.

A short review routine

Together, the two views give you a review that fits in ten minutes:

1. **Outline.** Read the headings only, and fix the skeleton first, since every later fix is wasted if a section is going to move.
2. **Visual read.** Read the whole document in the visual view as a reader would, and note what makes you stop.
3. **Source pass.** Fix your notes in the source, where you can see exactly what you are changing.
4. **Commit**, if the document is in git, with a message saying what the review changed.

The order matters: structure before sentences, reading before fixing, and the source as the final word on what the document contains.

The common thread

What ties all this together is keeping the document honest. Plain text means the file is just words and a few marks, readable by anyone and owned by you. Writing prose before adding structure means the headings and lists describe an argument you have actually made. Version control keeps every change visible and reversible, with the decisions logged. The two views let you check that what you see on the page is what is in the file.

None of this needs MD Studio. The habits work in any text editor with a git folder behind it. What MD Studio adds is the two views side by side, so checking structure costs a glance at the other pane instead of a switch to another program. I want a writing tool to make the good habits cheap and otherwise stay out of the way, and your documents should outlast it.

Now you try

Pick a document you actually have to write this week, not an exercise.

1. Write the first draft as prose, with no headings or lists, until you run out.
2. Read it back and mark the seams. Turn them into headings, and try to make each one say something.
3. Open the outline (Ctrl+Shift+O) and read the skeleton on its own. Fix what it shows you.
4. Read the whole thing once in the visual view, then fix what you noticed in the source.
5. If you use git, commit at step 1 and again at step 4, then look at the diff between the two commits. That diff is the clearest picture you'll get of what your editing did to the draft.

The appendices that follow are reference material, and the next document you write is where the practice happens.

Appendix A: Keyboard Reference

This appendix lists every shortcut in MD Studio, grouped by what you are doing. The app-wide shortcuts come first. After them come the keys that belong to each editor pane, because the Markdown pane and the visual pane are different editors with different habits.

Three notes apply throughout:

- **On macOS, Cmd replaces Ctrl.** Ctrl+S is Cmd+S, Ctrl+K is Cmd+K, and so on.
- **Ctrl+1 to Ctrl+4 work in the installed app.** In a normal browser tab they switch browser tabs instead, so use the toolbar or the command palette there. Installing MD Studio is covered in Chapter 1.
- **Ctrl+W and Ctrl+Tab always belong to the browser.** That is why closing and switching MD Studio's own tabs uses Alt instead.

If you remember only one shortcut, make it **Ctrl+K**. The command palette lists every command by name, with its shortcut beside it, so it teaches you the rest.

Files

Shortcut	Action
Ctrl+S	Save. Writes back to the open file in Edge and Chrome; downloads elsewhere. A new document asks where to save.
Ctrl+Shift+S	Save as…. Always asks for a name and location.
Ctrl+O	Open file…
Alt+N	New document
Ctrl+P	Print the formatted document (the browser's print dialog).

Saving, opening and the difference between Edge and Chrome and other browsers are covered in Chapter 5. There is no shortcut for **Export**; use the toolbar, or type "export" in the command palette (Chapter 6).

Tabs

Shortcut	Action
Alt+W	Close tab. Asks first if the tab has unsaved changes.
Alt+PageDown	Next tab
Alt+PageUp	Previous tab

With a tab in the tab bar focused (click it, or Tab to it):

Key	Action

Left / Right arrow	Move to the previous or next tab.
Shift+Left / Shift+Right	Move the current tab one place left or right.
Middle click	Close that tab.

You can also reorder tabs by dragging them.

Views and panels

Shortcut	Action
Ctrl+1	Split view: Markdown source and formatted document side by side.
Ctrl+2	Markdown view: Markdown source only.
Ctrl+3	Visual view: formatted document only.
Ctrl+4	Read view: formatted document only, not editable, laid out for reading.
Ctrl+Shift+O	Show or hide the Outline.
Ctrl+,	Open Settings.

Linked scroll has no shortcut. Toggle it in Settings (**Scroll both panes together in split view**) or with *Toggle linked scroll* in the command palette. **?** shows every shortcut in the app, whenever no text is being typed.

Reading

These work in Read view only, when no dialog is open and no text field has focus.

Key	Action
J or K	Scroll down or up a little.
Space or Shift+Space	Page down or up.
G or Shift+G	Jump to the start or the end.
[or]	Previous or next heading.
/	Open the find bar (without replace).
F	Focus mode: only the page. Escape brings the toolbar back.
?	List these keys.
Alt+Left or Alt+Right	Back or Forward through the notes you opened by following links or from the folder panel.

In the folder panel, the arrow keys move between notes: Right opens a folder and then enters it, Left closes it and then climbs out, Home and End jump to the first and last row, and Enter opens a note.

Command palette

Shortcut	Action
Ctrl+K or Ctrl+Shift+P	Open the command palette.
Up / Down arrow	Move through the list.
Enter	Run the highlighted command.
Escape	Close the palette.

Type to filter; the match is forgiving, so `exp pdf` finds *Export as PDF*. Start with `#` to list the document's headings instead of commands, then type a few letters and press Enter to jump to that section. *Go to heading…* does the same.

Find and replace

Shortcut	Action
Ctrl+F	Find: opens the find bar above the editors.
Ctrl+H	Find and replace: the find bar with the replace field open.
Enter (in the find field)	Next match.
Shift+Enter (in the find field)	Previous match.
Enter (in the replace field)	Replace the current match.
Escape	Close the find bar.

The bar searches the pane you are working in. Its option buttons, **Match case** (Aa), **Whole word** (W) and **Regular expression** (.*), are toggled with the mouse or by tabbing to them.

Editing in both panes

Shortcut	Action
Ctrl+Z	Undo, in the pane that has focus.
Ctrl+Y or Ctrl+Shift+Z	Redo.
Ctrl+V	Paste. Pasting an image file saves it next to the document or embeds it (Chapter 5).
Ctrl+A	Select all.

Each pane keeps its own undo history. An edit you make in the visual pane is undone from the visual pane, not from the Markdown pane, and the other way round.

The visual pane

These shortcuts come from the visual editor and work while the cursor is in the formatted

document. The formatting toolbar above it does the same jobs with the mouse.

Shortcut	Action
Ctrl+B	Bold
Ctrl+I	Italic
Ctrl+E	Inline code
Ctrl+Alt+X	Strikethrough
Ctrl+Alt+0	Turn the block into a Paragraph.
Ctrl+Alt+1 to Ctrl+Alt+6	Turn the block into a heading, level 1 to 6.
Ctrl+Alt+7	Numbered list
Ctrl+Alt+8	Bulleted list
Ctrl+Shift+B	Quote
Ctrl+Alt+C	Code block
Shift+Enter	Line break inside a paragraph.
Tab or Ctrl+]	Indent a list item one level.
Shift+Tab or Ctrl+[	Outdent a list item one level.

Inside a table:

Shortcut	Action
Tab or Ctrl+]	Next cell.
Shift+Tab or Ctrl+[	Previous cell.
Ctrl+Enter	Leave the table.

On keyboard layouts where Ctrl+Alt doubles as AltGr, such as Danish, German or Norwegian on Windows, some Ctrl+Alt combinations are also how you type characters like `{` and `[`. If a combination types a character instead of formatting, or the other way round, use the formatting toolbar.

The Markdown pane

The Markdown pane is a code editor, and it has a code editor's keys. The ones most useful for writing:

Shortcut	Action
Enter	New line. In a list or block quote, continues it with the next -, 1. or >.
Ctrl+] / Ctrl+[	Indent / outdent the current line or selection.
Alt+Up / Alt+Down	Move the current line (or selected lines) up or down.
Shift+Alt+Up / Shift+Alt+Down	Copy the current line above or below.

Ctrl+Shift+K	Delete the current line.
Ctrl+Enter	Insert a blank line below, wherever the cursor is in the line.
Ctrl+D	Select the next occurrence of the selected word, for editing several at once.
Ctrl+Shift+L	Select every occurrence of the selection.
Ctrl+Alt+Up / Ctrl+Alt+Down	Add a cursor on the line above or below.
Alt+drag	Select a rectangular block of text.
Ctrl+/	Wrap the line in an HTML comment (<!-- … -->), or remove it.
Ctrl+U	Undo the last cursor movement or selection change.
Escape	Collapse several cursors or selections back to one.

Two differences from the visual pane are worth knowing:

- **Ctrl+B does nothing, and Ctrl+I selects** the enclosing piece of Markdown instead of italicising. In the source you type the marks yourself: `**bold**`, `*italic*`.
- **Tab moves focus out of the pane** rather than inserting a tab character, so keyboard users can always leave the editor. Use Ctrl+] to indent.

Dialogs and messages

Key	Action
Escape	Close the open dialog (Settings, Export, a confirmation) without acting.
Enter	Confirm the focused button.
Tab / Shift+Tab	Move between the controls in a dialog.

The short list

If this is too much to take in at once, start with these and let the command palette teach you the rest:

Shortcut	Action
Ctrl+K	Command palette: any action, by name.
Ctrl+S	Save.
Ctrl+O	Open.
Ctrl+F	Find.
Ctrl+Shift+O	Outline.
Ctrl+B / Ctrl+I	Bold / italic (visual pane).

Ctrl+Z	Undo.

Appendix B: Markdown Syntax Reference

This appendix lists all the Markdown MD Studio understands. MD Studio reads CommonMark, the core specification, plus the GitHub Flavored Markdown (GFM) extensions, and its visual pane also renders footnotes. For when and why to use each construct, see Chapter 3 and Chapter 4.

Each entry gives the source as you type it in the Markdown pane and what the visual pane shows. Where there is one, it also names the format toolbar control or shortcut that produces the same thing in the visual pane. Entries that say "type" followed by some characters are typing shortcuts the visual editor is set up to convert as you type.

Headings

Source	Result	In the visual pane
# Title	Level 1 heading	Text style -> Heading 1, or type #
## Section	Level 2 heading	Heading 2, or type ##
### Subsection	Level 3 heading	Heading 3, or type ###
#### Part	Level 4 heading	Heading 4, or type ####
##### Minor	Level 5 heading	Type #####
###### Least	Level 6 heading	Type ######

A space after the hash marks is required: `#tag` is an ordinary paragraph. Optional closing hashes (`## Section ##`) are ignored.

The first two levels also have the *setext* form, text underlined on the next line:

```
Title
=====

Section
-------
```

The outline, the # heading jump in the command palette and linked scroll all use top-level ATX (#) and setext headings. They do not count headings inside code blocks, block quotes or list items.

Paragraphs and line breaks

Source	Result
Text separated by a blank line	Two paragraphs
Two lines with no blank line between them	One paragraph; the line break becomes a space
A line ending in a backslash \	A hard line break inside the paragraph
A line ending in two or more spaces	A hard line break (invisible in the source, so prefer the backslash)

Emphasis and inline formatting

Source	Result	In the visual pane
italic or _italic_	italic	Italic, Ctrl+I
bold or __bold__	bold	Bold, Ctrl+B
both	both	Bold and Italic
~~struck~~ (GFM)	struck	Strikethrough
`code`	code	Inline code
`` a `tick` ``	a `tick`	

Markdown leaves underscores inside a word alone, so `file_name_here` stays as written. Nothing inside inline code is formatted.

Links

Source	Result	In the visual pane
[text](https://example.com)	A link reading text	Link, then the address and Apply
[text](https://example.com "Title")	The same link with a tooltip	
[text](other-file.md)	A relative link to a file in the same folder	Link
<https://example.com>	The address itself as a link	
<someone@example.com>	An email link	
www.example.com or https://example.com in text (GFM)	Turned into a link automatically	

To remove a link in the visual pane, put the cursor in it, click **Link**, clear the address and choose **Apply**.

Reference links keep long addresses out of the paragraph. The definition lines do not render:

```
Read the [lease][lease] and the [terms][].

[lease]: https://example.com/lease
[terms]: https://example.com/terms
```

Images

Source	Result
![Alt text](images/plan.png)	The image from a file next to the document
![Alt text](https://example.com/plan.png)	The image from the web
![Alt text](images/plan.png "Title")	The image with a title

![Alt text](<my plan.png>)	A file name with spaces, in angle brackets (my%20plan.png also works)
![Alt text|300](images/plan.png)	The image shown 300 pixels wide; |300x200 sets both sides

In that last row a pipe and a number at the end of the alt text set the width. The alt text itself is kept as written, and the size is not read as part of the description. Obsidian writes sizes the same way.

Pasting (**Ctrl+V**) or dropping a file writes the link for you. In Edge or Chrome with the document saved on disk, MD Studio saves the file in the folder the vault's settings name (an `images` folder next to the document when there is no `.obsidian` folder) and links it, as a wikilink or a Markdown link depending on **Link style** in Settings. Elsewhere an image is shrunk to at most 1600 px and embedded as a `data:` address, and other kinds of file are not inserted. See Chapter 4 and the next section.

Obsidian embeds and wikilinks

Notes written in Obsidian use double brackets to embed and link. MD Studio keeps these exactly as written: the visual pane shows each one as a picture, a player or a labelled chip, and an edit in either pane never rewrites the token. The files come from the attachment folder you choose, which should be the vault root (Chapter 5). Embeds work in Edge and Chrome; in other browsers they appear as labelled placeholders and the source is unchanged.

Source	Result
![[plan.png]]	The image named plan.png, found anywhere in the vault
![[plan.png|300]]	The image 300 pixels wide, height following
![[plan.png|300x200]]	The image 300 by 200 pixels
![[plan.png|Floor plan]]	The image with alt text; |Floor plan|300 gives both
![[report.pdf]]	The PDF in a frame where the browser allows it, with an Open link beside it
![[report.pdf#page=3&height=400]]	The PDF opened at page 3, 400 pixels high
![[talk.mp3]]	An audio player
![[demo.mp4|480]]	A video player 480 pixels wide
![[data.zip]]	A labelled chip with an Open link
![[Meeting notes]]	The whole note, shown as a quoted block you cannot edit
![[Meeting notes#Decisions]]	The section under that heading
![[Meeting notes#^abc123]]	The paragraph, list item or quote that ends in ^abc123; a lone ^abc123 line after a list names the whole list
![[#Decisions]] and ![[#^abc123]]	The same, from the note you are editing

[[Meeting notes]] and [[Meeting notes|the notes]]	A plain chip showing the name or the alias

A few rules sit behind the table. A size is read from the last option that is digits or digits, an x and digits, and only for images, PDFs and video; for a note, an audio file or a link, a number is an alias, so `[[Note\|2024]]` shows 2024. File kinds follow the extension, and a name with no extension, or with one that is not a known file type, such as `Note v1.2`, is a note.

A bare name is looked up across the whole vault. If two files share it, the one nearest the note wins and the tooltip says which. A name with a slash must match the end of a path, and `./` or `../` is relative to the note. Wikilinks are not percent-decoded: write the space, not `%20`. A Markdown link such as `![](images/x.png)` is looked up next to the note first, then across the vault.

A web address in an embed, as in `![[https://example.com/a.png]]`, is a web image: it follows **Load web images** in Settings, in the visual pane and in exports, and a size after the `|` is ignored. An embed of a section of the note you are editing, or of a web address, needs no folder, so it does not raise the attachment bar.

Transcluded notes can go three levels deep, and a note never repeats itself inside itself, so a loop shows a short notice instead. A section runs from its heading to the next heading of the same or a higher level, sub-headings included. Front matter is left out, and so are the `^abc123` markers, which belong to the source. Links inside a transcluded note are shown as plain text unless they are web or email addresses. A note larger than 2 MB is not embedded; the placeholder says so.

Inside a table, write the pipe in an embed as a backslash and a pipe, as in the rows above, or the table breaks. In the visual pane a chip that holds a bare pipe, because it was copied or dragged into a cell or edited with Find and Replace, is written to the file with the pipe escaped.

Some things are not embeds, and are left alone: anything in a fenced or indented code block (also inside a list item or a quote), in a code span (also one that wraps over a line break) or in front matter, a token whose opening bracket has a backslash in front of it, an empty `![[]]`, and a token that runs over a line break. A backslash before the `!` only, as in `\![[a.png]]`, leaves the `!` as text and makes the rest a plain wikilink. In the visual pane, turning a paragraph into a code block, or putting inline code around a token, changes the chip back to its text, so the token is never hidden inside code. Undoing that with the opposite action (turning the block back into a paragraph, or removing the inline code) makes the token a chip again.

MD Studio has some known gaps. Only top-level headings can be named after the `#` in a transclusion, not headings inside a list, a quote or a callout. A token that Markdown splits into pieces, such as `![[_name_]]` with emphasis or a bare web address inside the name, stays plain text and may gain backslashes if you edit that paragraph in the visual pane. The same is true of a token inside the label of a Markdown link, as in `[see ![[plan.png]]](https://example.com)`: it is shown as text, and editing that paragraph in the visual pane writes the brackets with backslashes (edit such a link in the Markdown pane). Obsidian's `%%comments%%` are not understood. Clicking a wikilink does nothing yet, and there is no link autocomplete.

Lists

Source	Result	In the visual pane
Lines starting - , * or +	Bulleted list	Bulleted list, or type -
Lines starting 1. (or 1))	Numbered list	Numbered list, or type 1.
A first item numbered 4.	Numbered list starting at 4	
- [ ] Task (GFM)	Task list item with an empty box	
- [x] Done (GFM)	Task list item with a ticked box	

Nested lists are indented to line up with the parent item's text:

```
1. Pack
   - Label boxes
   - Essentials box
2. Move
   - [ ] Book the van
   - [x] Tell the landlord
```

Only the number of the first item counts, and the rest follow in order, so a list written `1.`, `1.`, `1.` renders as 1, 2, 3. A list with blank lines between its items is *loose*, and each item is spaced as a paragraph. Task boxes change by editing `[ ]` and `[x]` in the Markdown pane.

Block quotes

Source	Result	In the visual pane
> Quoted text	A quotation set off from the text	Quote, or type >
>> Nested	A quote inside a quote	

Mark blank lines inside a quote with > to keep several paragraphs in it:

```
> First paragraph of the quote.
>
> Second paragraph.
```

Code blocks

A fenced code block, with an optional language hint after the opening fence:

````
```json
{ "rooms": 12 }
```
````

Source	Result	In the visual pane
Lines between ``` fences	Code block, shown exactly as written	Code block, or type ``` and a space
Lines between ~~~ fences	The same	

``python as the opening fence	Code block with a language hint; the Markdown pane colours the code	
Lines indented four spaces	Indented code block (no language hint)	

A fence of four backticks can contain a line of three, which is how you show a code block inside a code block.

Horizontal rules

Source	Result	In the visual pane
---, *** or ___ on a line of its own	A horizontal line	Horizontal rule

Leave a blank line above `---`. Directly under a line of text it makes a level-2 setext heading instead.

Tables (GFM)

```
| Option         | Cost  | Ready by |
| :------------- | ----: | :------: |
| Stay and refit | 1.2 M | June     |
| Harbour site   | 1.8 M | March    |
```

Separator cell	Column alignment
--- or :---	Left
---:	Right
:---:	Centred

The first row is the header, and the second row of dashes is required. The pipes do not need to line up, and the outer pipes are optional. Cells hold inline content only (emphasis, links and code), so there are no lists, line breaks or merged cells inside a table. Write a literal pipe inside a cell as `\|`. In the visual pane, **Insert table** adds a three by three table.

Footnotes

```
The site floods about once a decade.[^flood]

[^flood]: Municipal climate report, 2025.
```

The marker `[^label]` goes in the text. The definition `[^label]: ...` can sit anywhere, usually at the end. Labels can be numbers or words. Footnotes render in MD Studio's visual pane and on GitHub, but they are not part of CommonMark, and MD Studio's exports currently show them as the plain text you wrote.

Escaping

A backslash before a punctuation character shows the character instead of its Markdown meaning.

Source	Result
not italic	*not italic*
\# not a heading	# not a heading
1986\. A good year.	1986. A good year. (not a numbered list)
\[not a link\](x)	[not a link](x)

Characters that can be escaped: `\ ` * _ { } [ ] ( ) # + - . ! |` and the other ASCII punctuation marks.

Raw HTML

CommonMark allows HTML tags in the text. Avoid them in MD Studio documents. They are unreadable as plain text, different tools treat them differently, and MD Studio does not render them. HTML exports show them as text.

CommonMark or GFM at a glance

Construct	CommonMark	GFM adds
Headings, paragraphs, line breaks	Yes	
Emphasis, strong, inline code	Yes	Strikethrough
Links, images, <...> autolinks	Yes	Bare-address autolinks
Bulleted and numbered lists	Yes	Task lists
Block quotes, code blocks, rules	Yes	
Tables		Yes
Footnotes		Supported by GitHub; rendered in MD Studio's visual pane

Appendix C: Further Reading

This book is deliberately practical. It has enough Markdown to write well in MD Studio and enough method to make plain text pay off. If you want to go further into the format, into working with plain text and version control, or into writing itself, these are the places I would look, with a note on what each is good for.

On Markdown itself

John Gruber, *Markdown* (daringfireball.net/projects/markdown). The original description, published in 2004 with help from Aaron Swartz. The syntax page is still worth reading, more for its philosophy than for its rules. Gruber wanted Markdown source to read as plain text without looking marked up, and that goal is why the format has lasted. The page is informal and leaves many edge cases open, which is why the specifications below exist.

The CommonMark specification (commonmark.org, spec at spec.commonmark.org). A precise, tested definition of core Markdown, written to settle the ambiguities in Gruber's original so that the same file renders the same way in different tools. It is long and exact, and you won't read it cover to cover. Go to it when something renders in a way you didn't expect, such as a list that won't nest or a line break that doesn't appear. The spec explains the behaviour and gives an example for every rule. The site also has a short interactive tutorial for newcomers.

The GitHub Flavored Markdown specification (github.github.com/gfm). GFM is CommonMark plus the extensions most people now expect: tables, task lists, strikethrough and automatic links. It is written as a strict superset of the CommonMark spec, with the extensions marked. If you share Markdown on GitHub, or anywhere that follows its conventions, this is the definition that applies. Chapter 4 and Appendix B note which syntax is core and which is GFM.

The Markdown Guide (markdownguide.org). A friendly, free reference with a basic syntax page, an extended syntax page and a cheat sheet. Its notes on which applications support which features help when a document has to move between tools. Start here if the specifications feel like too much.

On converting and publishing

Pandoc (pandoc.org). A free command-line tool that converts between dozens of document formats, including Markdown to and from Word, HTML and LaTeX, and Markdown to PDF and EPUB. MD Studio's own exports (Chapter 6) cover the everyday cases. Pandoc is where to go when you need more, such as citations and a bibliography, a custom Word template, or a book assembled from many chapter files. Its user's guide is long but well organised, and it documents Pandoc's own Markdown dialect, which has many useful extensions. Pandoc's author, John MacFarlane, is also one of the authors of the CommonMark specification.

On plain text and version control

Semantic Line Breaks (sembr.org). A short specification of the convention described in Chapter 7:

break lines at sentences and natural pauses so that diffs show meaningful changes. It fits on one page, explains its reasoning and collects the history of the idea. Read it before you decide whether the convention is worth adopting for your own documents.

Brandon Rhodes, "Semantic Linefeeds". The blog post that brought the idea to a modern audience. Rhodes traces it back to advice Brian Kernighan gave early Unix users: start each new sentence on a new line, so that edits stay local. It is a quick read, and it shows the technique is much older than git.

Scott Chacon and Ben Straub, *Pro Git* (git-scm.com/book). The standard introduction to git, free to read online and translated into many languages. Chapters 1 and 2 are all you need to commit, read history and compare versions of your documents. The chapter on branching comes next, when you want to try alternatives as described in Chapter 7. You can skip everything about servers and team workflows until you need it.

Kieran Healy, *The Plain Person's Guide to Plain Text Social Science*. An essay by a sociologist on why and how to do academic writing in plain text, with Markdown, Pandoc and version control. It is written for researchers, but its case for plain text, and its frank account of what it costs, applies to anyone with documents they want to keep.

On writing

None of these books mention Markdown. They are about the writing itself, which the format is there to serve.

Barbara Minto, *The Pyramid Principle*. The classic method for structuring business writing: start with the answer, group the supporting points beneath it, and order each group logically. It treats in full the idea from Chapter 7 that headings should state claims, since an outline that reads as an argument is a pyramid seen from above. It is dense and repays a slow read, especially before you write a recommendation or a report.

William Zinsser, *On Writing Well*. A guide to clear non-fiction, and a long argument against clutter. Its chapters on simplicity and on rewriting pair well with version control, because Zinsser's advice to cut is much easier to follow once deleting is safe.

Anne Lamott, *Bird by Bird*. A book about the writing life, and the best-known case for the bad first draft. If Chapter 7's advice to write prose first and structure later feels uncomfortable, read her chapter on first drafts. She is also very funny.

Steven Pinker, *The Sense of Style*. A modern style guide grounded in linguistics and psychology. Its discussion of the "curse of knowledge", the difficulty of imagining what your reader doesn't already know, explains why reading your own document in the visual view, the way a reader would, catches so much.

On the tool

MD Studio is open source. The application code is licensed under Apache 2.0 and this book under Creative Commons BY-NC 4.0. The live app, the repository and the project's dashboard are linked

from md-studio.struktureretsundfornuft.dk. If a chapter here describes something the app no longer does in exactly that way, the repository is the source of truth, and a good place to report the difference.

The shortest possible reading list

If you read only one more thing, make it Gruber's syntax page. It takes ten minutes, it explains why Markdown looks the way it does, and everything else on this list builds on it. After that, write something real; you will learn more from a week of using the format than from more reading.